

DANH SÁCH LIÊN KẾT ĐƠN (SINGLY LINKED LIST)

I. TỔNG QUAN	2
1.1 Biến tĩnh	2
1.2. Biến động	2
1.3. Danh sách	2
1.3.1 Các phép toán trên danh sách	2
1.3.2 Danh sách đặc	2
1.3.3 Hai hình thức tổ chức cơ bản của danh sách là mảng (liên kết ngầm) và danh sách liên kết (liên kết tường minh)	3
II. DANH SÁCH LIÊN KẾT (LINKED LIST)	5
2.1 Tính chất	5
2.2 Cấu trúc dữ liệu của 1 nút trong DSLK đơn	6
2.3 Giải thích ý nghĩa của cấu trúc node	6
2.4 Tạo con trỏ head và con trỏ tail	6
2.5 Các giải thuật trên DSLK đơn	7
2.5.1 Tạo một danh sách liên kết đơn rỗng	7
2.5.2 Tạo một phần tử mới	7
2.5.3 Thêm một phần tử vào danh sách	7
2.5.3.1 Thêm phần tử vào đầu DSLK	7
2.5.3.2 Thêm phần tử vào cuối DSLK	8
2.5.3.3 Thêm phần tử vào sau một phần tử của DSLK	10
2.5.3.4 Tìm phần tử trong danh sách liên kết	11
2.5.3.5 Duyệt danh sách	14
2.5.3.6 Hủy một phần tử trong danh sách	15
2.5.3.7 Quản lý số lượng phần tử trong danh sách	20
2.5.3.8 Xóa danh sách liên kết đơn	21
2.5.3.9 Sắp xếp danh sách liên kết đơn	22
III. CÁC CẤU TRÚC ĐẶC BIỆT CỦA DANH SÁCH LIÊN KẾT ĐƠN	26
3.1 Stack (ngăn xếp)	26
3.1.1 Cài đặt bằng mảng 1 chiều	26
3.1.2 Cài đặt bằng danh sách liên kết	32
3.2 Queue (hàng đợi)	37
3.2.1 Cài đặt bằng mảng 1 chiều	38
3.2.2 Cài đặt bằng danh sách liên kết đơn	43

I. TỔNG QUAN

1.1 Biến tĩnh

- Là biến được khai báo tường minh, có tên gọi, tồn tại trong phạm vi khai báo.
- Biến tĩnh có kích thước không đổi => không tận dụng hiệu quả bộ nhớ.

1.2. Biến động

- Là biến không được khai báo tường minh, không có tên gọi, có thể xin khi cần, giải phóng khi sử dụng xong.
- Biến động linh động về kích thước => tận dụng hiệu quả bộ nhớ.
- Đề thao tác với biến động, ta lưu địa chỉ của nó bằng biến con trỏ => truy xuất biến động thông qua biến con trỏ.

1.3. Danh sách

Là một tập hợp rỗng hoặc gồm nhiều phần tử (element) $a_1, a_2, \dots, a_n$ mà tính chất cấu trúc của nó là mối liên hệ tương đối giữa các phần tử với nhau: *nếu biết được phần tử a_i thì ta sẽ biết được vị trí của phần tử a_{i+1} .*

- Danh sách là một kiểu dữ liệu dạng tuyến tính, nó bao gồm tập hợp các phần tử có cùng kiểu dữ liệu. Mỗi phần tử trong danh sách có nhiều nhất 1 phần tử đứng trước và nhiều nhất 1 phần tử đứng sau.
- **Biểu diễn: $a_1, a_2, \dots, a_n$ với $n \geq 1$.**
 - Số các phần tử n gọi là chiều dài của danh sách (length).
 - $n = 0$: Danh sách rỗng (empty list).
 - Các phần tử có thể được sắp xếp tuyến tính theo vị trí của chúng trong danh sách: a_i đứng trước phần tử a_{i+1} với $i = 1, 2, \dots, n-1$ và phần tử a_i đứng sau phần tử a_{i-1} với $i = 2, 3, \dots, n$.
 - Ta nói rằng, phần tử a_i ở vị trí thứ i .

1.3.1 Các phép toán trên danh sách

- **Tìm kiếm một phần tử:** cần biết thông tin khóa (key). Kết quả trả về vị trí của phần tử nếu tìm thấy (true) hoặc vị trí 0 nếu không tìm thấy (false).
- **Thêm một phần tử:** Thêm đầu, giữa, cuối danh sách.
- **Lược bỏ một phần tử:** Trước khi thực hiện lược bỏ, cần thực hiện tìm kiếm phần tử cần lược bỏ, nếu không tìm thấy thì không thực hiện được phép này, ngược lại thì thực hiện phép lược bỏ và số phần tử trong danh sách giảm đi 1.
- **Thay thế một phần tử của danh sách bằng một phần tử khác:** Số phần tử của danh sách không thay đổi, nhưng có thể bị thay đổi vị trí nếu thỏa mãn một điều kiện ràng buộc (xếp theo ABC, tăng dần...).
- **Tách 1 danh sách thành nhiều danh sách:** Tổng số phần tử của các danh sách mới phải bằng số phần tử của danh sách ban đầu.
- **Ghép nhiều danh sách thành 1 danh sách mới:** Đi sau các phần tử của danh sách thứ nhất sẽ là các phần tử của danh sách thứ hai, ... Sau đó, số phần tử của danh sách mới bằng tổng số phần tử của các danh sách ghép vào.
- **Trộn nhiều danh sách thành một danh sách mới:** Dựa vào một quy tắc nào đó (trộn 2 danh sách có cùng thứ tự để cho ra một danh sách có cùng thứ tự đó). Sau khi trộn, số phần tử của danh sách mới phải bằng số phần tử của các danh sách được trộn với nhau.
- **Sao chép một danh sách sang một danh sách mới:** Kết quả là một danh sách mới (đích) giống hệt hoàn toàn danh sách ban đầu (nguồn).
- **Sắp xếp thứ tự một danh sách:** Phải dựa vào một khóa nào đó. Phép sắp thứ tự sẽ làm thay đổi vị trí của các phần tử sao cho khóa của phần tử này có thứ tự. Phép này thực hiện ngay trên chính danh sách đó.

1.3.2 Danh sách đặc

$a[1]$	
$a[2]$	
	$\vdots$
$a[n]$	

- ☐ Là một danh sách mà các phần tử được sắp xếp kế tiếp nhau trong bộ nhớ, đứng ngay sau vị trí của phần tử a_i là vị trí của phần tử của a_{i+1} .
- ☐ Nếu các phần tử có chiều dài bằng nhau thì ta sử dụng công thức tính địa chỉ.

Gọi: $a[1]$: địa chỉ của phần tử đầu tiên.
 $a[i]$: địa chỉ của phần tử thứ i .
 d : chiều dài của một phần tử.
 Thì: $a[i] = a[1] + (i - 1) * d$.

- Các loại thuật toán:

- **Khởi tạo danh sách:** Khi khởi tạo, danh sách rỗng, ta cho số phần tử n bằng 0.
- **Thêm một phần tử vào danh sách:** Giả sử ta thêm vào một phần tử có nội dung NewInfo ngay tại vị trí thứ i , khi đó các phần tử $a[i]$ đến $a[n]$ được di chuyển lên các vị trí sau.
- **Loại bỏ một phần tử của danh sách:** Giả sử ta loại bỏ phần tử $a[i]$ của danh sách, khi đó các phần tử từ $a[i + 1]$ đến $a[n]$ được di chuyển đến một vị trí trước.
- **Tìm kiếm một phần tử trong danh sách:** Dùng giải thuật “tìm kiếm tuần tự” (đối với danh sách chưa có thứ tự hoặc “tìm kiếm nhị phân” (đối với danh sách đã có thứ tự)

➤ **Đặc điểm:** Mật độ sử dụng bộ nhớ của danh sách là tỉ số giữa số byte ghi nhớ thông tin chia cho tổng số byte của danh sách. Do đó mật độ của danh sách đặc là 100%.

Ưu điểm	Nhược điểm
- Không lãng phí bộ nhớ (mật độ cao nhất (100%). - Dễ dàng truy xuất phần tử $a[i]$ bằng công thức tính địa chỉ. - Dễ dàng tìm kiếm một phần tử bằng tìm kiếm nhị phân.	- Không phù hợp cho phép thêm vào và loại bỏ. - Thời gian thực hiện các phép thêm vào hoặc phép loại bỏ phụ thuộc vào số phần tử của danh sách và vị trí thực hiện phép thêm vào hoặc phép loại bỏ. - Theo tính toán, trung bình khoảng nửa danh sách bị di chuyển đi một vị trí khi thêm hoặc loại bỏ một phần tử.

1.3.3 Hai hình thức tổ chức cơ bản của danh sách là mảng (liên kết ngầm) và danh sách liên kết (liên kết tường minh)

Mảng	DSLK
Mọi liên hệ giữa các phần tử được thể hiện ngầm: • $A[i]$: phần tử thứ $i+1$ trong danh sách. • $A[i]$ và $A[i+1]$ là các phần tử kế cận trong danh sách.	Cấu trúc dữ liệu cho mỗi phần tử trong danh sách liên kết là 1 cấu trúc tự trở bao gồm: • Thông tin bản thân phần tử; • Địa chỉ của phần tử kế trong danh sách. Mỗi phần tử của danh sách liên kết là một biến động.

a) Mảng:



Bộ nhớ được cấp phát cho mảng là một khối bộ nhớ liên tiếp nhau, các phần tử trong mảng có thể truy cập thông qua chỉ số với độ phức tạp $O(1)$.

Ưu điểm

- + Đơn giản và dễ sử dụng.
- + Truy cập mảng với độ phức tạp là hằng số.

Nhược điểm

- Có thể gây lãng phí bộ nhớ nếu không sử dụng hết bộ nhớ xin cấp phát cho mảng.
- Kích thước mảng là cố định.
- Bộ nhớ cấp phát theo khối.
- Việc chèn và xóa phần tử khó khăn.

b) DSLK:



Bộ nhớ cấp phát cho các node trong DSLK có thể nằm rải rác nhau trong bộ nhớ.

Ưu điểm

- + Có thể mở rộng với độ phức tạp là hằng số.
- + Dễ dàng mở rộng và thu hẹp kích thước.
- + Có thể cấp phát số lượng lớn các node tùy vào bộ nhớ.

Nhược điểm

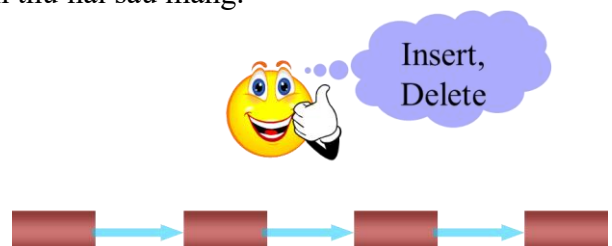
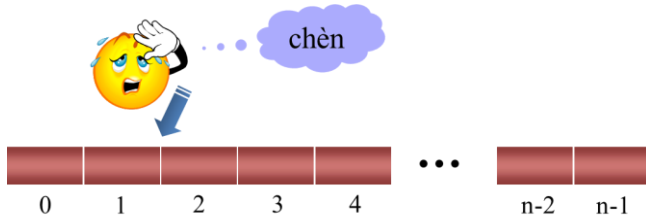
- Khó khăn trong việc truy cập một phần tử ở vị trí bất kỳ ($O(n)$).
- Khó khăn trong việc cài đặt.
- Tốn thêm bộ nhớ cho phần tham chiếu bổ sung.

Nội dung	Mảng	Danh sách liên kết
Kích thước	- Kích thước cố định - Cần chỉ rõ kích thước trong khi khai báo	- Kích thước thay đổi trong quá trình thêm/ xóa phần tử - Kích thước tối đa phụ thuộc vào bộ nhớ
Cấp phát bộ nhớ	Tĩnh: Bộ nhớ được cấp phát trong quá trình biên dịch	Động: Bộ nhớ được cấp phát trong quá trình chạy
Thứ tự & sắp xếp	Được lưu trữ trên một dãy ô nhớ liên tục	Được lưu trữ trên các ô nhớ ngẫu nhiên
Truy cập	Truy cập tới phần tử ngẫu nhiên trực tiếp bằng cách sử dụng chỉ số mảng: $O(1)$	Truy cập tới phần tử ngẫu nhiên cần phải duyệt từ đầu/cuối đến phần tử đó: $O(n)$
Tìm kiếm	Tìm kiếm tuyến tính hoặc tìm kiếm nhị phân	Chỉ có thể tìm kiếm tuyến tính

- Độ phức tạp của các thao tác trong mảng và DSLK

Thao tác	DSLK	Mảng
Truy xuất phần tử	$O(n)$	$O(1)$
Chèn/Xóa ở đầu	$O(1)$	$O(n)$ nếu mảng chưa full
Chèn ở cuối	$O(n)$	$O(1)$ nếu mảng chưa full
Xóa ở cuối	$O(n)$	$O(1)$
Chèn giữa	$O(n)$	$O(n)$ nếu mảng chưa full
Xóa giữa	$O(n)$	$O(n)$

- Danh sách liên kết được xem là 1 cấu trúc dữ liệu cơ bản, được sử dụng để khắc phục hạn chế của mảng (như việc mảng bị cố định về kích thước) và được sử dụng phổ biến thứ hai sau mảng.



II. DANH SÁCH LIÊN KẾT (LINKED LIST)

- Danh sách liên kết là một danh sách mà các phần tử được nối kết với nhau nhờ vào vùng liên kết của chúng, **vùng liên kết của phần tử a_i chứa địa chỉ của phần tử a_{i+1} .**
- Là một loại cấu trúc đơn giản và thích hợp với các phép thêm, loại bỏ, ghép nhiều danh sách...
- **Là cấu trúc dữ liệu dùng để lưu trữ một danh sách (tập hợp hữu hạn) dữ liệu. Điểm đặc biệt của cấu trúc này là khả năng chứa của nó **động** (có thể mở rộng và thu hẹp dễ dàng). Vì vậy có thể cấp bộ nhớ động cho danh sách liên kết (không giới hạn phần tử)**
- Để tổ chức một danh sách thích hợp cho các phép toán trên, ta dùng DSLK mà các phần tử của nó có thể chứa ở những vùng không kề nhau trong bộ nhớ và chúng được nối với nhau nhờ vào vùng liên kết: vùng liên kết của phần tử thứ nhất chứa địa chỉ của phần tử thứ hai, vùng liên kết của phần tử thứ hai chứa địa chỉ của phần tử thứ ba... vùng liên kết của phần tử cuối cùng là null.

2.1 Tính chất

-  DSLK có thể mở rộng và thu hẹp một cách linh hoạt.
-  Phần tử cuối cùng trong DSLK trở vào NULL.
-  Không lãng phí bộ nhớ nhưng cần thêm bộ nhớ để lưu phần con trỏ.
-  Các phần tử trong DSLK được gọi là Node, được cấp phát động.
-  Đây là CTDL cấp phát động nên khi còn bộ nhớ thì sẽ còn thêm được phần tử vào DSLK.

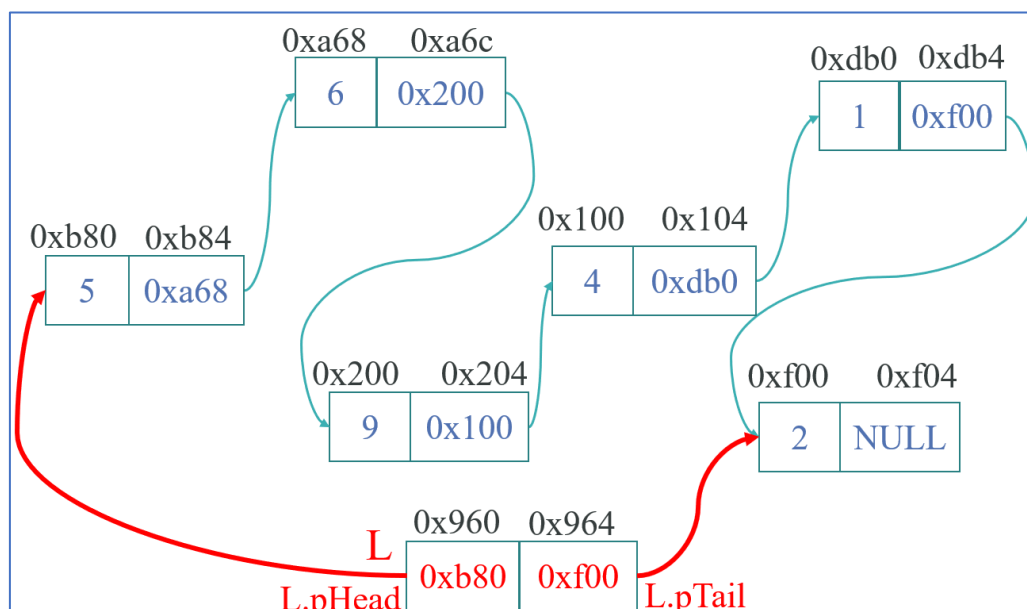
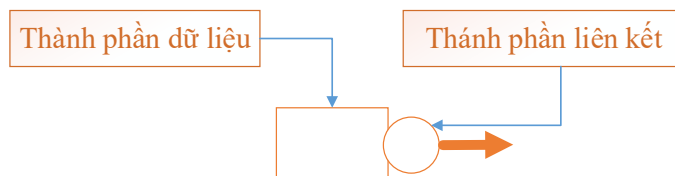
Danh sách liên kết đơn là một **tập hợp các Node được phân bố động**, được sắp xếp theo cách sao cho **mỗi Node chứa “một giá trị” (Data) và “một con trỏ”(Next)**. Con trỏ sẽ trỏ đến phần tử kế tiếp của danh sách liên kết đó. **Nếu con trỏ mà trỏ tới NULL, nghĩa là đó là phần tử cuối cùng của danh sách liên kết đơn.**



- **Toa đầu (Head node):** Toa đầu tượng trưng cho node đầu, con trỏ head sẽ trỏ đến node này.
- **Toa cuối (Tail node):** Toa cuối đại diện cho node cuối và node cuối trỏ đến NULL
- **Dây xích (Link):** các pointer (con trỏ next)

Như vậy, mỗi phần tử trong **danh sách liên kết đơn** là một cấu trúc có hai thành phần:

- **Thành phần dữ liệu (Data):** Lưu trữ thông tin về bản thân phần tử (*Họ tên, MSSV/ số nguyên, số thực/ Một sinh viên – struct...*);
- **Thành phần liên kết (Link):** (Pointer trong C++) Lưu **địa chỉ phần tử đứng sau trong danh sách hoặc bằng NULL** nếu là phần tử cuối danh sách.



2.2 Cấu trúc dữ liệu của 1 nút trong DSLK đơn

```
class element {
private:
    int data; //Phần dữ liệu của node
    element* next; //Để lưu địa chỉ của node tiếp theo trong danh sách liên kết
                  // và ta phải dùng con trỏ có kiểu element để lưu địa chỉ đó.
                  //pointer to another object of same type
};
```

Tuy nhiên nếu dùng class, thì ta không thể trực tiếp truy cập vào thuộc tính của element này được, cho nên ta phải tạo thêm các hàm tương ứng (constructor, destructor, getter, setter) để thay đổi nội dung của các thuộc tính trong element.

```
class element {
private:
    int data;
    element* pointer;
public:
    //Constructor
    element() { data = 0; next = nullptr; };
    element(int data1) { data = data1; next = nullptr; };

    //Destructor
    ~element() {};

    //Getter
    int getData() { return data; };
    element* getPointer() { return pointer; };

    //Setter
    void setData(int data2) { data = data2; };
    void setPointer(element* pointer) { next = pointer; };
};
```

Giả sử, nếu ta viết: `void setData(int data) { data = data; };`

Thì compiler sẽ hiểu `data` ở vế trái là tham số của hàm (do tên biến đặt trùng nhau), lúc này ta phải thêm con trỏ `this` để phân biệt tham số hàm với thuộc tính của `element`.

```
void setData(int data) { this->data = data; };
```

```
void setPointer(element* pointer)
```

 (field) `int element::data`
Phần dữ liệu của node

2.3 Giải thích ý nghĩa của cấu trúc node

- ✓ Node ở đây có phần dữ liệu là một số nguyên lưu ở `data`, ngoài ra nó còn có 1 phần con trỏ tới chính struct node.
- ✓ Phần này chính là địa chỉ của node tiếp theo của nó trong DSLK.
- ✓ Như vậy mỗi node sẽ có dữ liệu của nó và có địa chỉ của node tiếp sau nó. Đối với node cuối cùng trong DSLK thì phần địa chỉ này sẽ là con trỏ NULL.
- ✓ Cần nhớ rằng, mỗi node trong DSLK đều được cấp phát động.

2.4 Tạo con trỏ head và con trỏ tail

```
class linkedList {
private:
    element* head; //Chứa địa chỉ của node đầu tiên
    element* tail;
public:
    linkedList() { head = tail = nullptr; };
    ~linkedList() {};

    //Getter
    element* getHead() { return head; };
    element* getTail() { return tail; };

    //Setter
    void setHead(element* ptr) { head = ptr; };
    void setTail(element* ptr) { tail = ptr; };
};
```


2.5 Các giải thuật trên DSLK đơn

2.5.1 Tạo một danh sách liên kết đơn rỗng

Địa chỉ của nút đầu tiên và nút cuối cùng đều không có: Phần khai báo class linkedList ta đã tạo.

2.5.2 Tạo một phần tử mới

Để tạo phần tử mới ta thực hiện theo các bước sau đây:

- Dùng toán tử **new** xin cấp phát một vùng nhớ đủ chứa một phần tử của danh sách (trong hàm main).
- Nhập thông tin cần lưu trữ vào phần tử mới. Con trỏ pointer được đặt bằng NULL.

```
int main()
{
    element* e;
    e = new element(9); //Phần này constructor sẽ làm việc
    delete e;

    return 0;
}
```

2.5.3 Thêm một phần tử vào danh sách

Có 3 trường hợp:

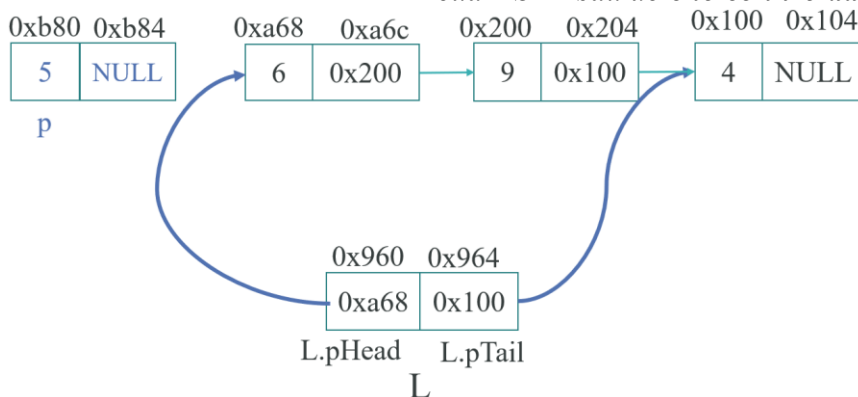
2.5.3.1 Thêm phần tử vào đầu DSLK

Khi thêm 1 phần tử vào List thì có làm cho head, tail thay đổi.

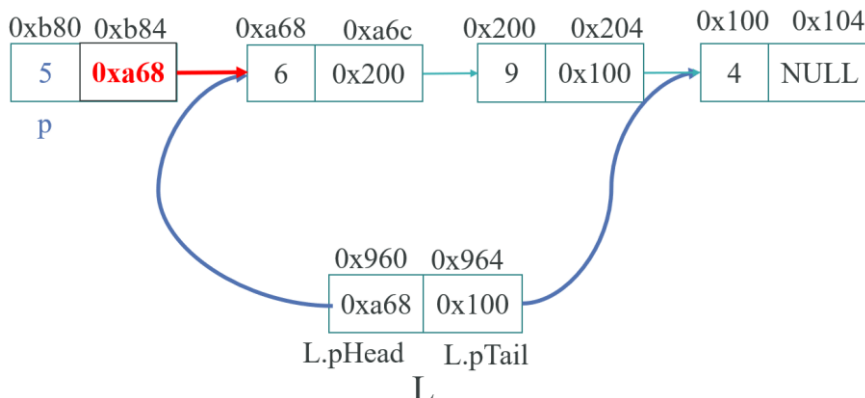
- ✓ **Thêm vào danh sách rỗng:** Lấy trỏ head, tail trỏ vào phần tử cần thêm vào.
- ✓ **Thêm vào danh sách đã có sẵn:**
 - + con trỏ của phần tử cần thêm trỏ vào phần tử đầu của danh sách.
 - + Lấy trỏ head trỏ vào phần tử cần thêm vào.

Các bước thực hiện như sau:

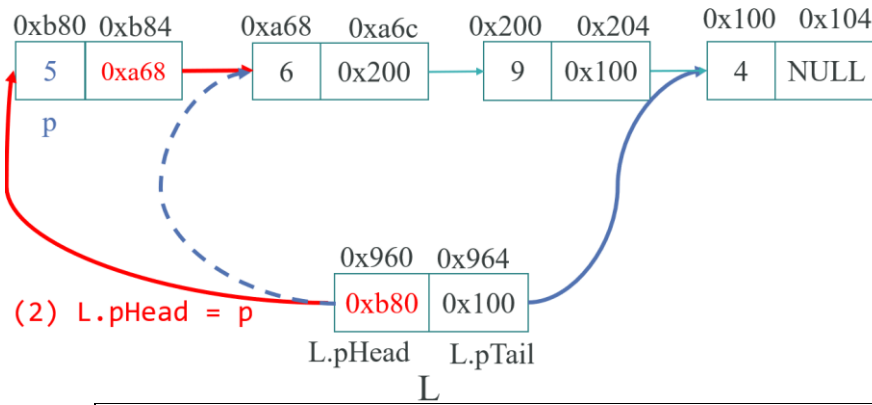
- Tạo và cấp phát bộ nhớ cho 1 nút mới.
- Nếu DSLK rỗng thì **gán phần tử đầu và cuối của DSLK bằng chính nút mới tạo.**
- Nếu DSLK khác rỗng thì cho **con trỏ next của nút mới** trỏ đến phần tử đầu tiên của DSLK sau đó **cho con trỏ đầu của DSLK** trỏ vào nút mới tạo.



(1) $p \rightarrow pNext = L.pHead$



(1) $p \rightarrow pNext = L.pHead$



(2) $L.pHead = p$

```
class linkedList {
...
public:
...
    void insertFirst(element* e) { //Xem lại bài con trỏ để hiểu bản chất
        //Kiểm tra danh sách có rỗng hay không -> Nếu đầu danh sách đi kiểm tra

        //Nếu danh sách rỗng
        if (head == nullptr) {
            setHead(e); setTail(e);
            //head = tail = e;
        }
        else {
            e->setPointer(head); //Gán next của e bằng địa chỉ của node đầu tiên
            setHead(e); //chuyển head trở đến phần tử mới.
        }
    };
};

int main()
{
    linkedList* list = new linkedList();
    element* e;
    e = new element(9); //e chứa địa chỉ của vùng nhớ được cấp phát bởi new
    list->insertFirst(e);
    e = new element(10);
    list->insertFirst(e);
    e = new element(13);
    list->insertFirst(e); //truyền đối số là địa chỉ của node e.

    list->Travel();

    delete e, list;

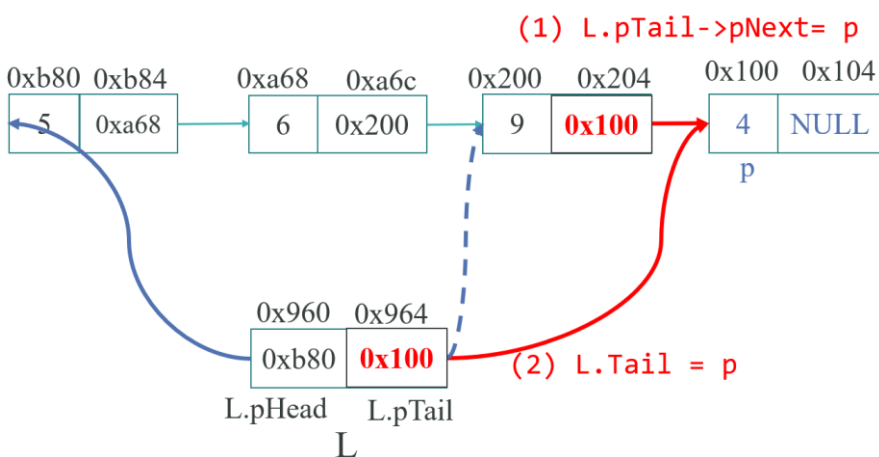
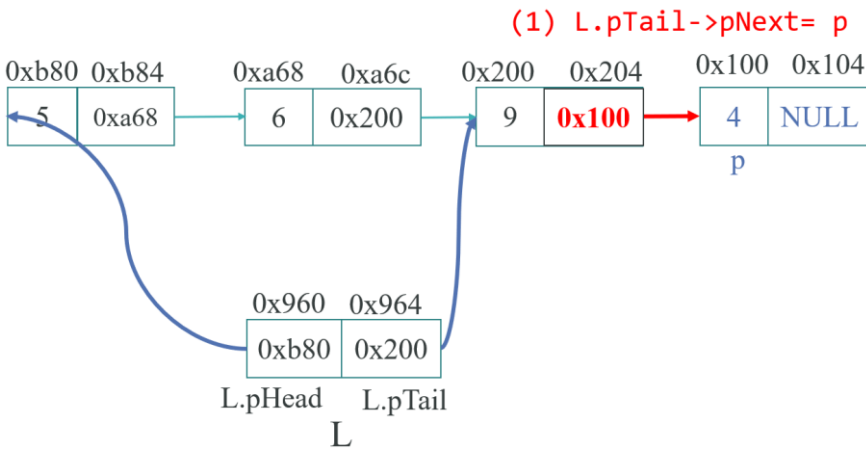
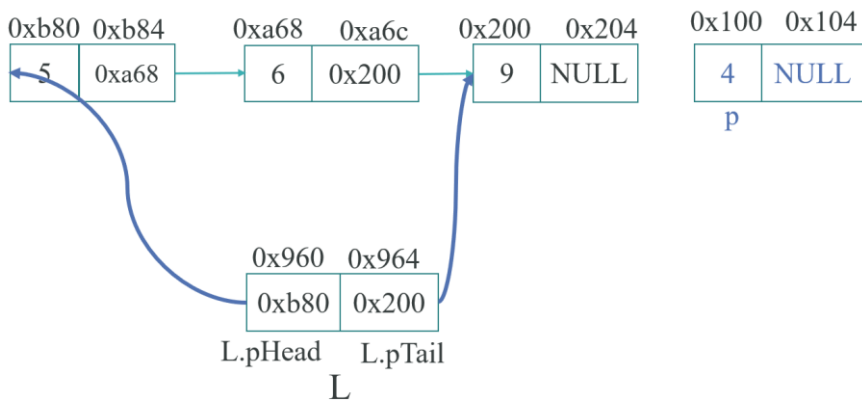
    return 0;
}
```

Kết quả: 13 10 9

2.5.3.2 Thêm phần tử vào cuối DSLK

Tạo và cấp phát bộ nhớ cho 1 nút mới.

- ✓ Nếu DSLK rỗng thì gán phần tử đầu và cuối của DSLK bằng nút mới tạo.
- ✓ Nếu DSLK khác rỗng:
 - Cho con trỏ **next** của **phần tử cuối** của DSLK cũ trỏ đến nút mới tạo.
 - Cho con trỏ **tail** trỏ vào node mới.



```
class linkedList {
...
public:
...
    void insertTail(element* e) {
        if (head == nullptr) {
            setHead(e); setTail(e);
        }
        else {
            tail->setPointer(e); //cho con trỏ next của node cuối trỏ vào node mới
            setTail (e);
        }
    };
};
```

```
int main()
{
    linkedList* list = new linkedList();
    element* e;
    e = new element(9); //e chứa địa chỉ của vùng nhớ được cấp phát bởi new
    list->insertTail(e);
    e = new element(10);
    list->insertTail(e);
    e = new element(13);
    list->insertTail(e);
}
```

```
list->Travel();

delete e, list;
return 0;
}
```

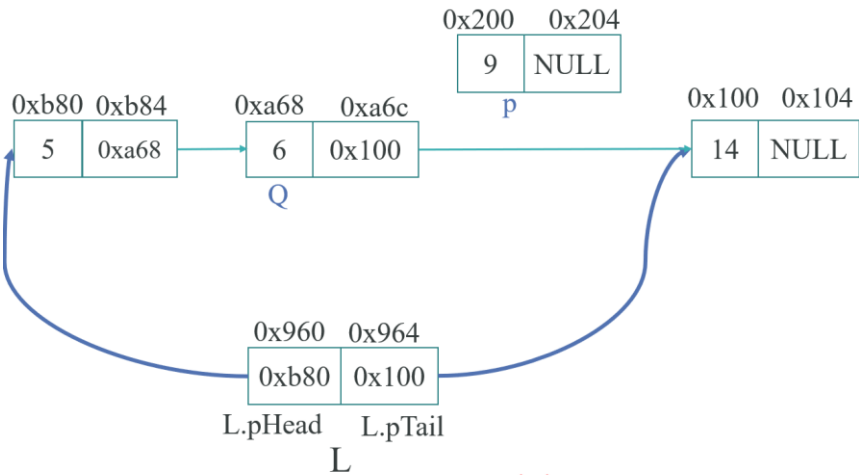
Kết quả: 9 10 13

2.5.3.3 Thêm phần tử vào sau một phần tử của DSLK

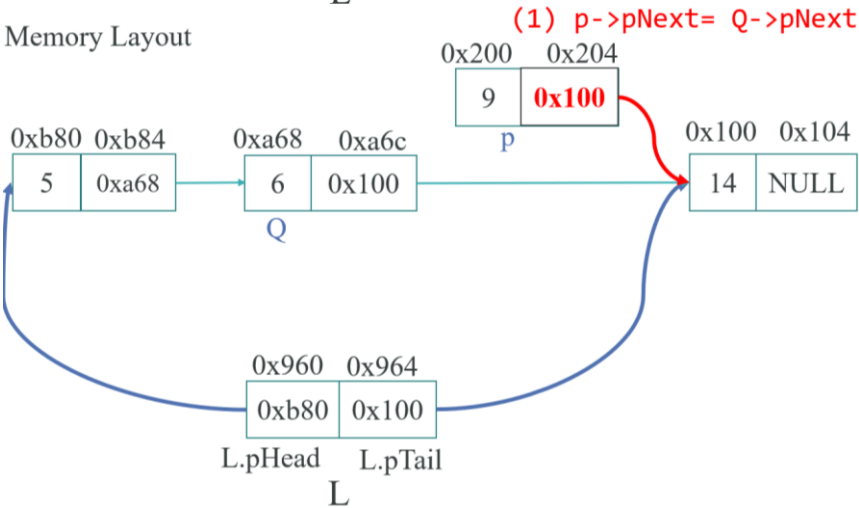
Các bước thực hiện như sau:

- + Tạo và cấp phát bộ nhớ cho 1 nút mới cần thêm.
- + Nếu tìm thấy nút q:
 - Cho con trỏ next của nút mới trở đến nút kế bên q.
 - Cho con trỏ next của q trở vào nút mới.
 - Trường hợp q là nút cuối thì **gán phần tử cuối của DSLK bằng nút mới thêm** (= Thêm phần tử vào cuối DSLK)
 - **Nếu không tìm thấy nút q** thì chèn nút mới vào đầu DSLK.

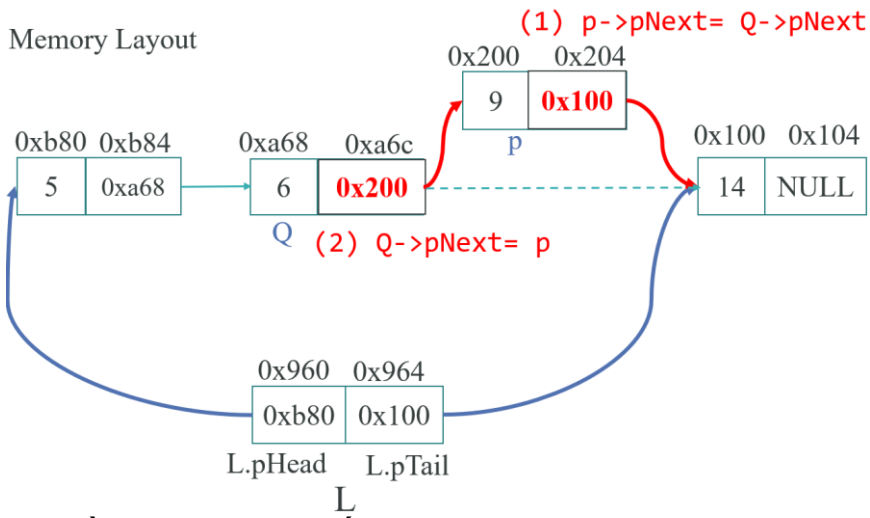
Ví dụ: Thêm 9 vào sau node Q có giá trị 6: {5, 6, 14}



Memory Layout

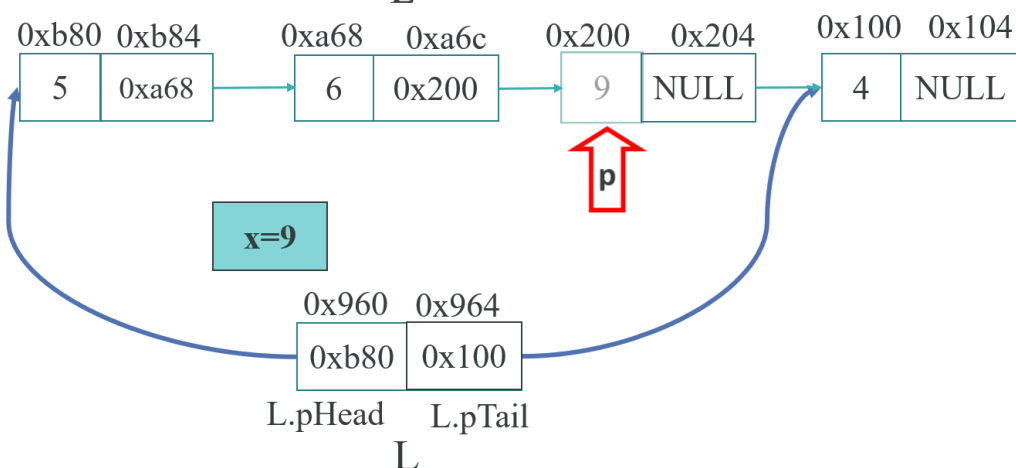
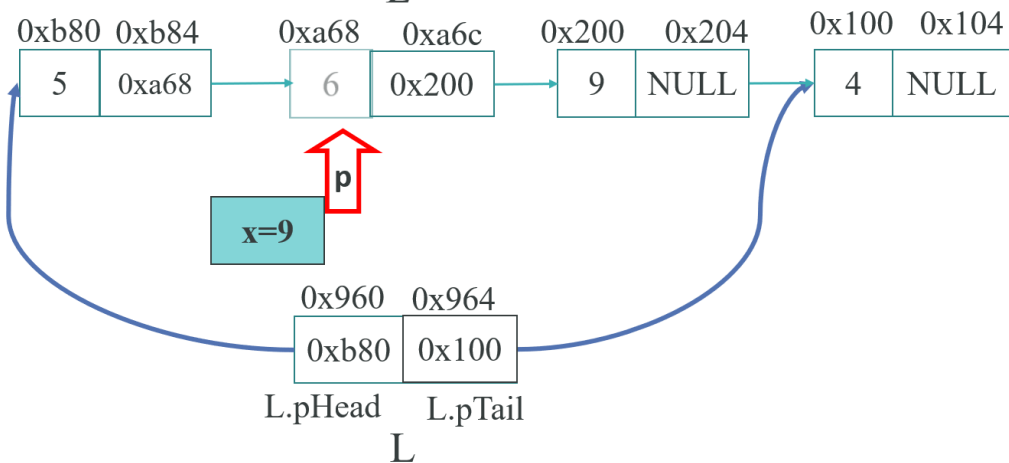
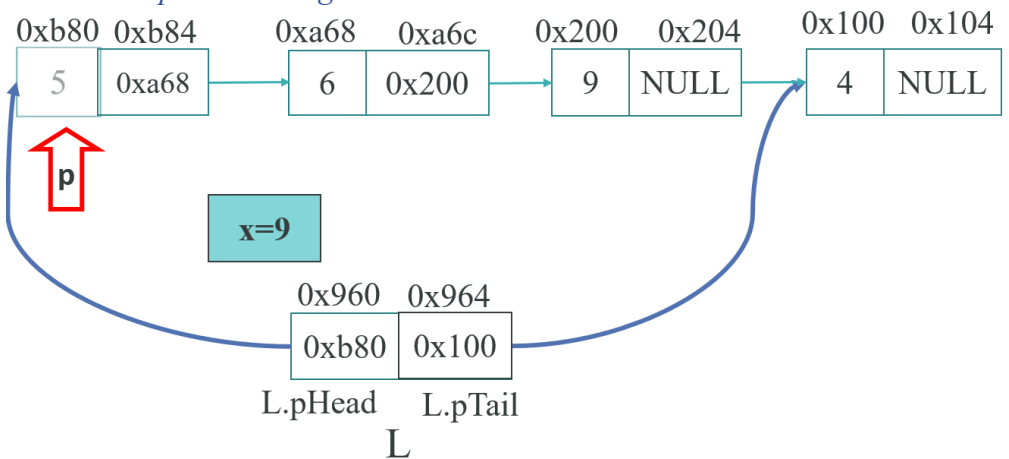


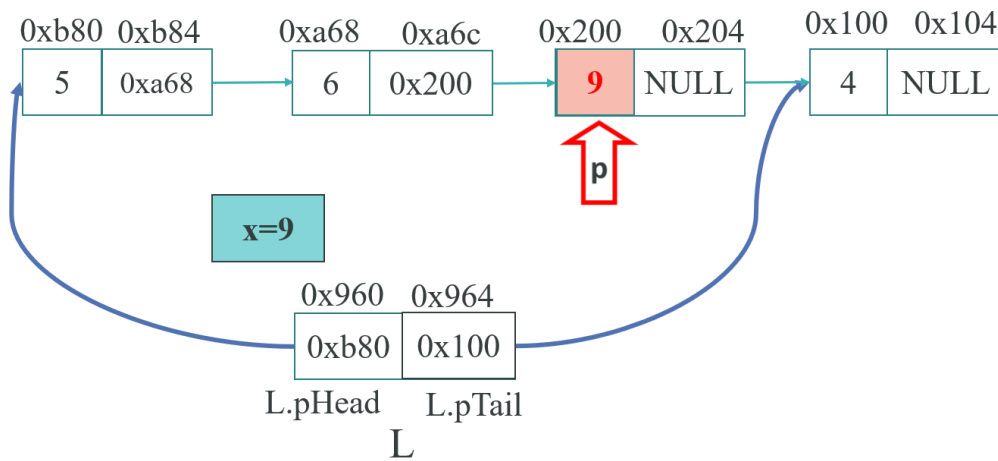
Memory Layout



* Ta cần một hàm tìm kiếm node có chứa data = X, sau đó ta triển khai hàm chèn giữa:

2.5.3.4 Tìm phần tử trong danh sách liên kết





```

class linkedList {
...
public:
...
    element* findX(int dat) { // trả về con trỏ p
        element* p = head;
        while (p != nullptr) {
            if (p->getData() == dat)
                return p;

            p = p->getPointer();
        }
    }

    // Thêm phần tử p vào sau phần tử q
    void insertBehind(element *p, int data3) {
        // p là node cần thêm, data3 do người dùng nhập vào

        element* q = findX(data3); // q là node cần tìm để chèn p phía sau
        if (q == nullptr) {
            cout << "Không tìm thấy phần tử chưa dữ liệu " << data3 << endl;
            cout << "Phần tử sẽ chèn ở đầu danh sách.\n";
            insertFirst(p); // nếu không tìm thấy node q thì chèn đầu
        }

        else {
            p->setPointer(q->getPointer()); // cho pNext trỏ đến node kế node q
            q->setPointer(p);

            if (p->getPointer() == nullptr) // nếu như chèn xong mà p là node cuối,
                // lúc này chuyển con trỏ tail trỏ đến node p
                tail = p;
        }
    }
};
  
```

```

int main()
{
    linkedList* list = new linkedList();

    element* e;
    e = new element(9); // e chứa địa chỉ của vùng nhớ được cấp phát bởi new
    list->insertTail(e);
    e = new element(10);
    list->insertTail(e);
    e = new element(13);
    list->insertTail(e);

    int dataF;
    cout << "Nhập data của node cần tìm: "; cin >> dataF;
    e = new element(15);
    list->insertBehind(e, dataF);
  
```

```

list->Travel();

delete e, list;
return 0;
}

```

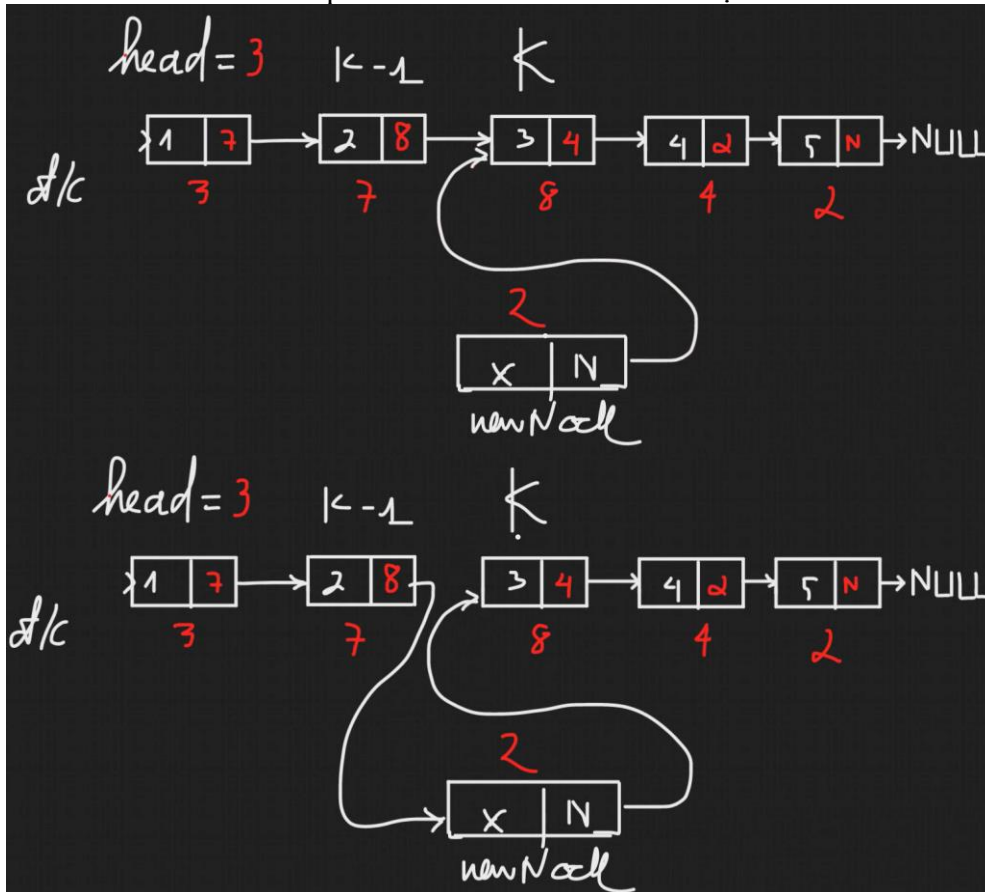
* Nếu ta cần chèn vào vị trí k bất kỳ trong danh sách:

- ✓ Nếu $K < 1$ hoặc K lớn hơn cỡ của DSLK thì vị trí chèn không hợp lệ.
- ✓ Nếu $K = 1$ thì ta thêm vào đầu.
- ✓ Nếu $K = (\text{số phần tử}) + 1$ thì ta thêm vào cuối.
- ✓ Ngược lại ta thực hiện theo 3 bước.

✚ **Bước 1:** Duyệt tới node trước vị trí cần chèn gọi là node $K - 1$.

✚ **Bước 2:** Cho con trỏ next của node mới lưu địa chỉ của node k .

✚ **Bước 3:** Cho phần next của node $K - 1$ lưu địa chỉ của node mới.



```

class LinkedList {
...
public:
...
    void insertPositionK(element* e, int k) {
        int n = countElement();
        if (k < 1 || k > n + 1) {
            cout << "Vi tri khong hop le. \n";
            return;
        } //vị trí không hợp lệ thì không làm gì cả
        else if (k == 1)
        {
            insertFirst(e);
            return;
        }
        else if (k == n + 1)
        {
            insertTail(e);
            return;
        }
    }
}

```

```

else { //cần k-2 vòng lặp để đến vị trí thứ k-1
    element* p = head;
    for (int i = 1; i <= k - 2; i++)
        p = p->getPointer(); //hết vòng lặp, p đang giữ địa chỉ node k-1

    e->setPointer(p->getPointer()); //cho node mới trỏ đến node k
    p->setPointer(e); //cho node k-1 trỏ đến node mới
}
};

```

```

int main()
{
    linkedList* list = new linkedList();

    element* e;
    e = new element(9); //e chứa địa chỉ của vùng nhớ được cấp phát bởi new
    list->insertTail(e);
    e = new element(10);
    list->insertTail(e);
    e = new element(13);
    list->insertTail(e);
    cout << "Danh sach hien co: ";
    list->Travel();
    cout << "\n=====\\n";

    int vitri;
    cout << "Nhap vi tri can chen node 30: "; cin >> vitri;
    e = new element(30);
    list->insertPositionK(e, vitri);
    cout << "\\nDanh sach hien co: ";
    list->Travel();

    delete e, list;
    return 0;
}

```

Kết quả:

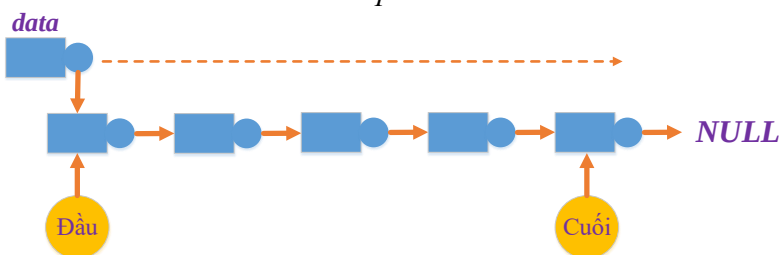
```

Danh sach hien co: 9    10    13
=====
Nhap vi tri can chen node 30: 4
Danh sach hien co: 9    10    13    30

```

2.5.3.5 Duyệt danh sách

- Ta sử dụng 1 biến con trỏ **tạm dạng nút trỏ vào phần tử đầu** của DSLK. Từ vị trí này, **theo liên kết giữa các nút** ta sẽ thực hiện việc duyệt qua từng phần tử trong DSLK.
- Trong quá trình duyệt, tại mỗi nút ta có thể thực hiện các thao tác như:
 - Lấy thông tin phần tử
 - Sửa thông tin phần tử
 - So sánh phần tử
 - Xóa phần tử...




```

class linkedList {
...
public:
...
    void Travel() {
        element* p = head;
        while (p != nullptr) {
            cout << p->getData() << "\t";
            p = p->getPointer(); //lấy địa chỉ của con trỏ của node hiện hành
        }
    };
};

```

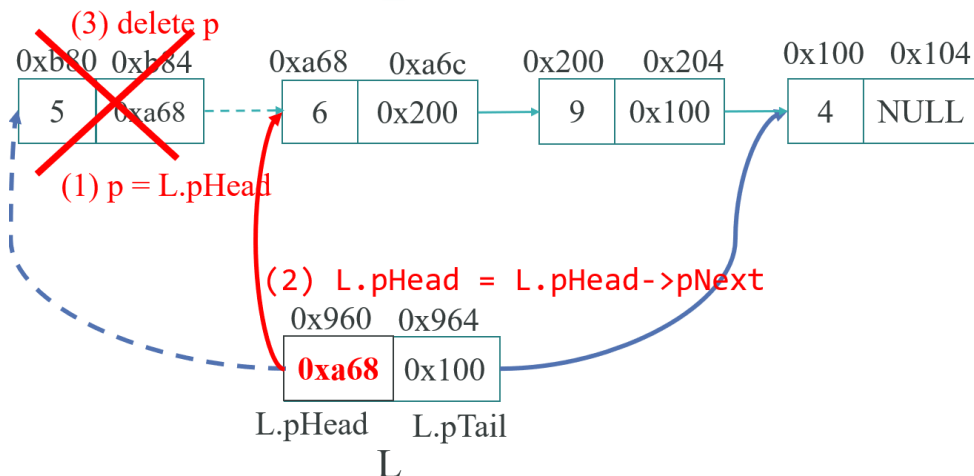
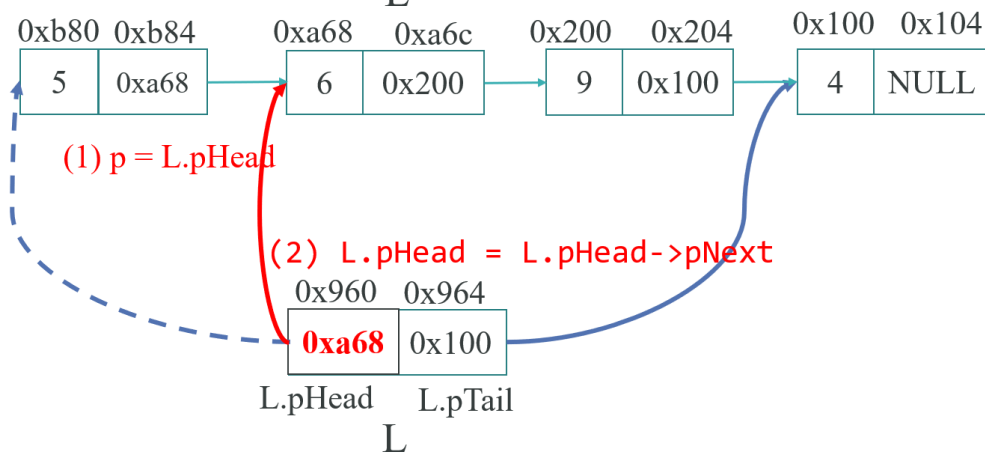
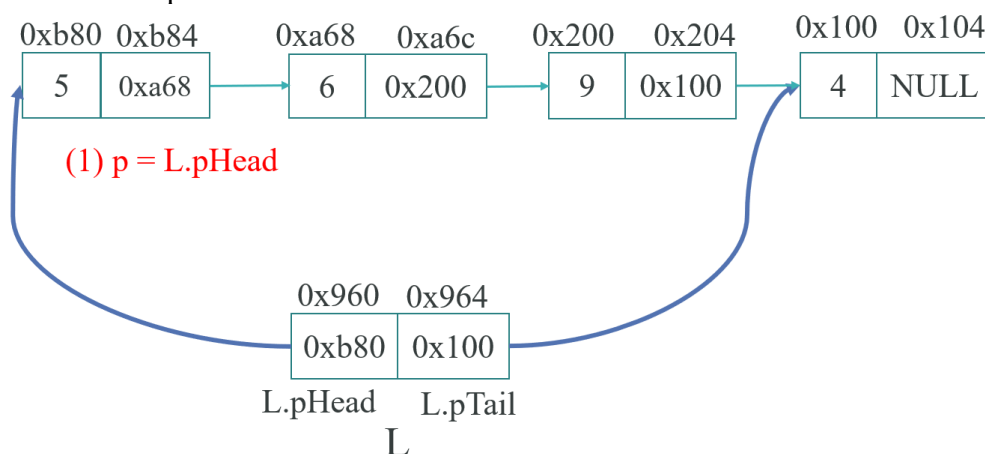
2.5.3.6 Hủy một phần tử trong danh sách

- Nếu DSLK chỉ có 1 node thì bạn cần xử lý riêng: Cho con trỏ head trỏ vào NULL

- Để xóa một nút p ra khỏi DSLK đơn ta làm như sau:

- Tìm nút q liền trước nút p.
- Nếu tìm thấy q, cho q liên kết với nút liền sau của p rồi giải phóng bộ nhớ đã cấp cho p.
- Nếu không tìm thấy q, tức p là nút đầu tiên, khi đó ta chỉ việc thay đổi nút đầu tiên của DSLK thành nút đứng liền sau p rồi thu hồi bộ nhớ của p.

➤ Xóa phần tử đầu danh sách



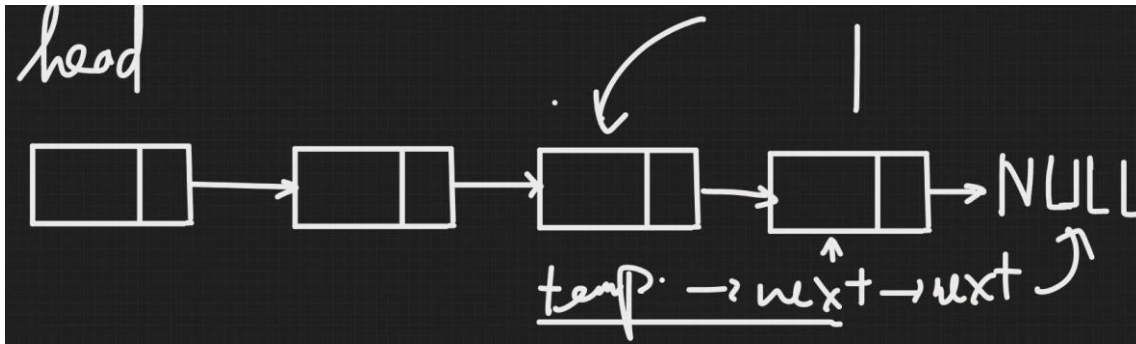
➤ **Xóa phần tử cuối trong danh sách**

Bước 1: Tìm tới node thứ 2 từ cuối về (node kế cuối) bằng con trỏ tmp.

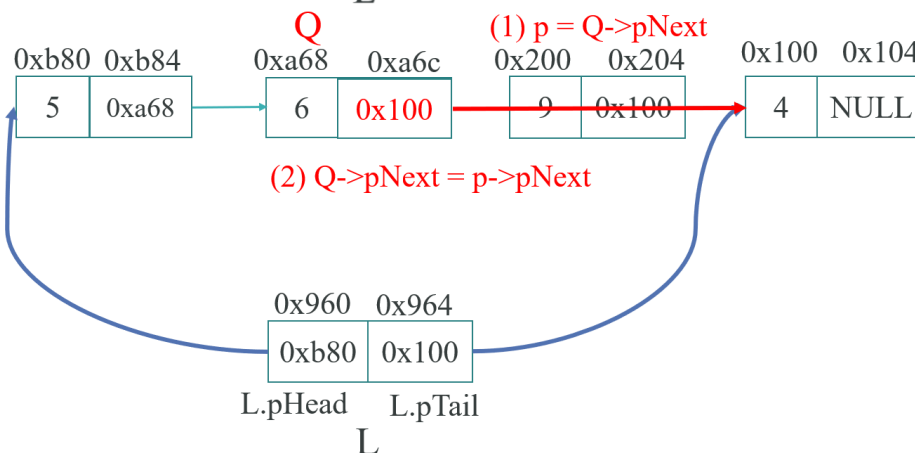
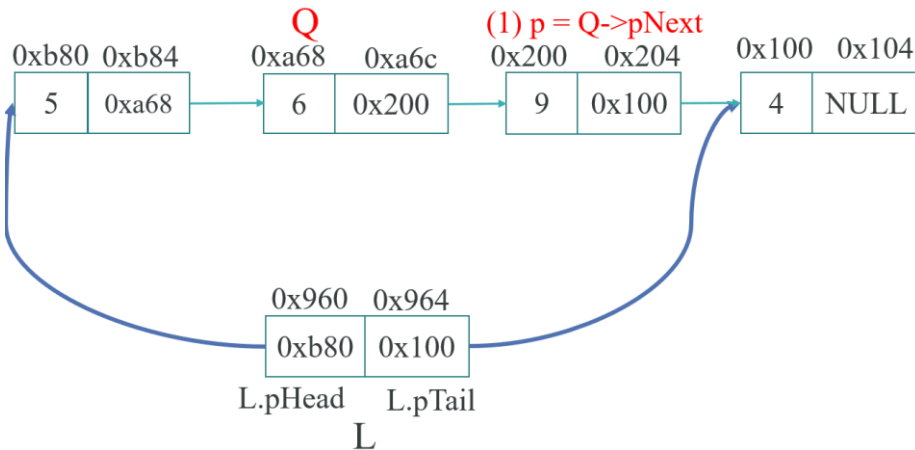
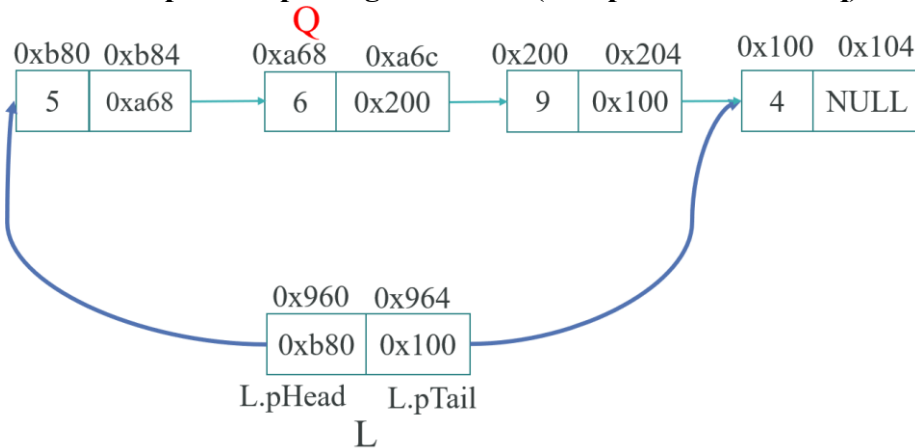
Bước 2: Cho phần next của node tmp trở vào NULL.

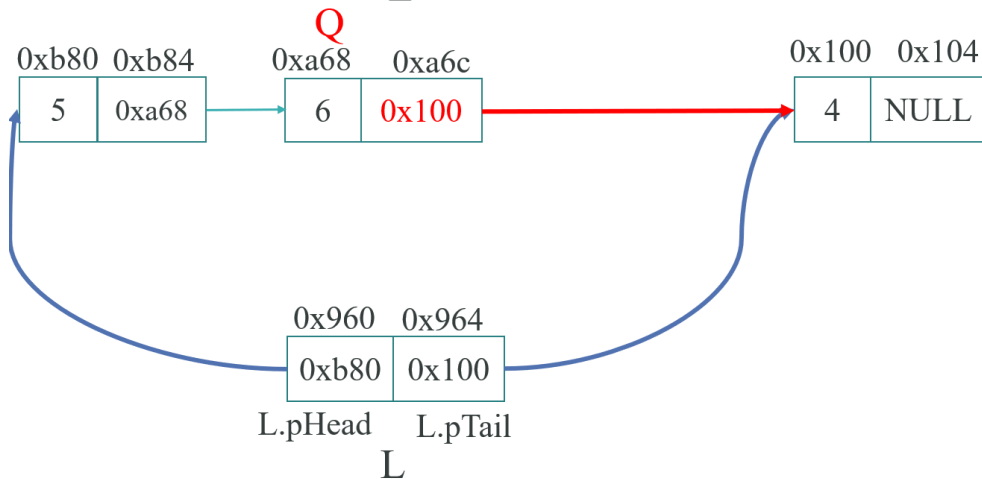
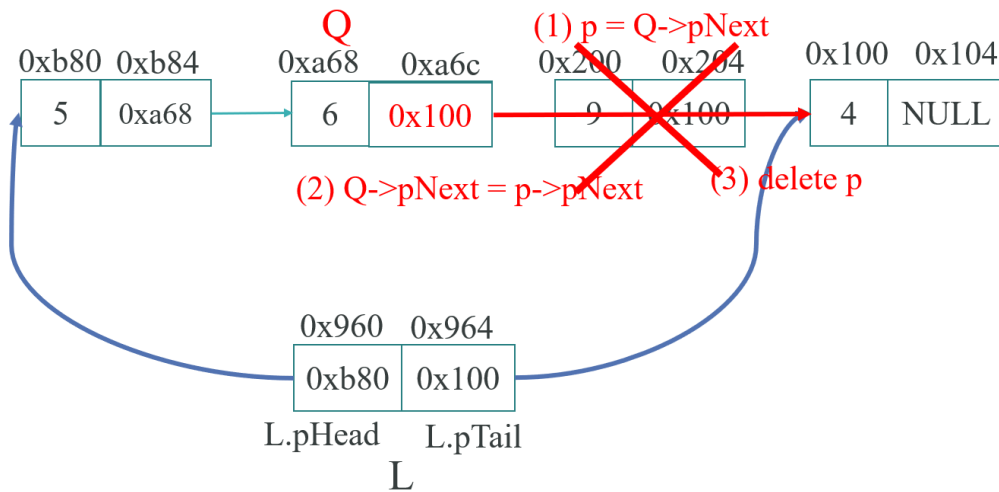
Bước 3: Giải phóng node cuối cùng trong DSLK.

✚ **Khi danh sách liên kết có nhiều hơn 1 node:**

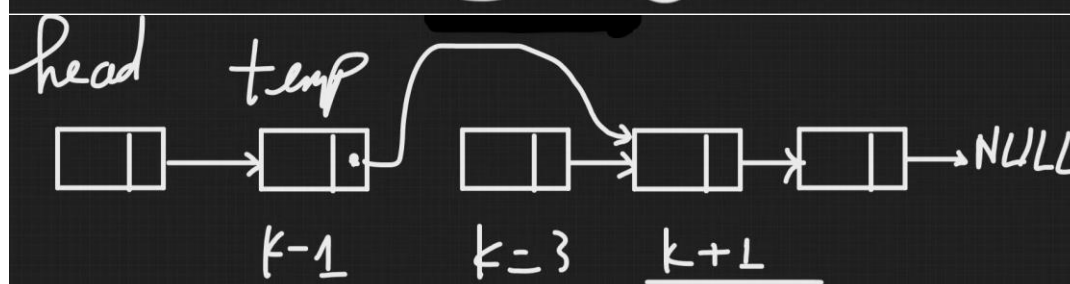
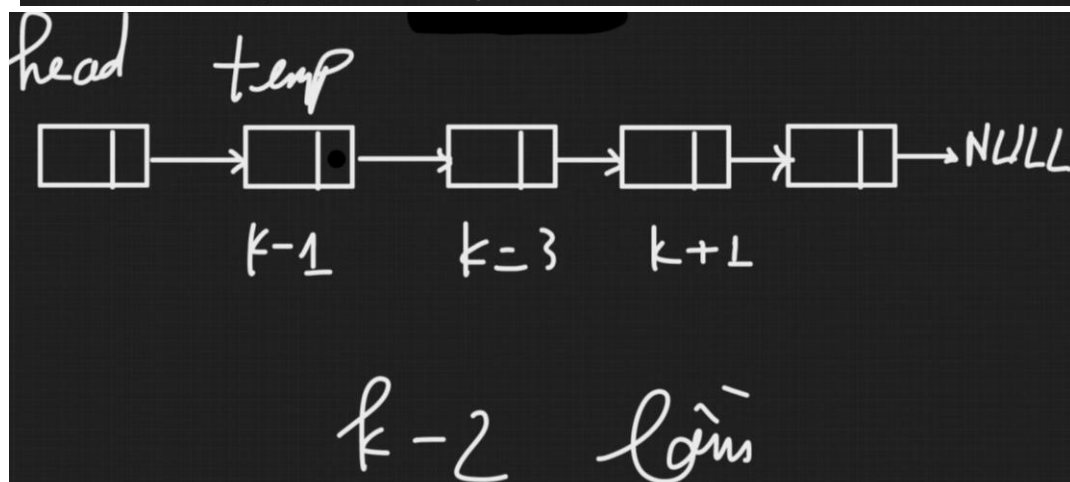
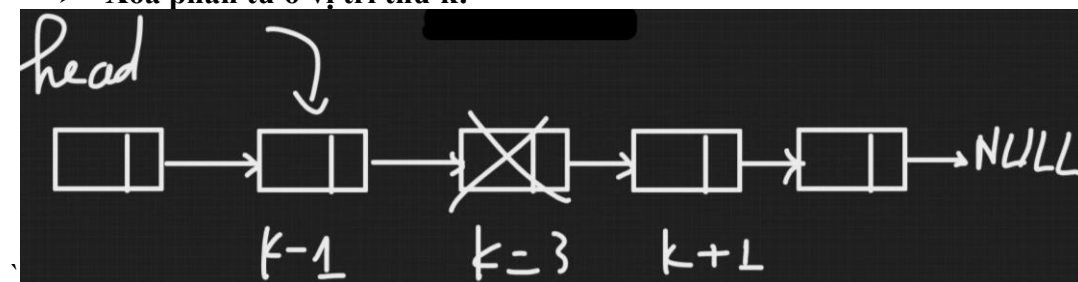


➤ **Xóa phần tử p trong danh sách (Xóa phần tử sau nút q)**





➤ Xóa phần tử ở vị trí thứ k:



```

class linkedList {
...
public:
...
    void deleteFirst() {
        element* p = head;
        if (head != nullptr) {
            if (p->getPointer() == nullptr) { //ds chỉ có 1 node
                setHead(nullptr); setTail(nullptr);
                return;
            }
            else
                head = p->getPointer();
            delete p;
        }
        else {
            cout << "Danh sach rong!";
            return;
        }
    }

    void deleteTail() {
        element* p = head;
        if (getHead() != nullptr) {
            if (p->getPointer() == nullptr) {
                deleteFirst(); //nếu ds chỉ 1 phần tử
            }
            else {
                while (p->getPointer()->getPointer() != nullptr) //quét đến node kế cuối
                {
                    p = p->getPointer(); //nếu như chưa đến node cuối thì p đi tiếp
                }
                element* xoa = p->getPointer(); //cho xoa là node cuối - node cần xóa
                setTail(p);
                p->setPointer(nullptr); //cho node kế cuối trở về null
                delete xoa;
            }
        }
        else {
            cout << "Danh sach rong! ";
            return;
        }
    }

    void deleteBehindQ(int dat1) {
        element* q = findX(dat1);
        if (q == nullptr) {
            cout << "Khong co phan tu " << q->getData() << endl;
        }
        else {
            element* p = q->getPointer();
            if (q->getPointer() == nullptr) {
                //giả sử q là phần tử cuối cùng hoặc dslk chỉ có đúng 1 phần tử là q
                cout << "Khong co phan tu ke node " << q->getData() << " de xoa.\n";
            }
            else {
                q->setPointer(p->getPointer()); //cho qNext trở về phần tử sau p
                delete p;
            }
        }
    }

    void deletePositionK(int k) {
        int n = countElement();
        if (k < 1 || k > n+1) {
            cout << "Vi tri khong hop le. ";
            return;
        }
    }
}

```

```

else if (k == 1) deleteFirst();
else if (k == n) deleteTail();
else {
    element* temp = head;
    for (int i = 1; i <= k - 2; i++) {
        temp = temp->getPointer(); //hết vòng lặp, p đang giữ địa chỉ node k-1
    }
    element* kXoa = temp->getPointer();

    //cho node k-1 kết nối với k+1
    temp->setPointer(kXoa->getPointer());
    delete kXoa;
}
}
};

```

```

int main()
{
    linkedList* list = new linkedList();
    element* e;
    e = new element(9); //e chứa địa chỉ của vùng nhớ được cấp phát bởi new
    list->insertTail(e);
    e = new element(10);
    list->insertTail(e);
    e = new element(13);
    list->insertTail(e);
    e = new element(15);
    list->insertTail(e);
    e = new element(18);
    list->insertTail(e);
    cout << "Danh sach hien co: ";
    list->Travel();
    cout << endl;

    cout << "=====MENU XOA=====\\n";
    cout << "0. Thoat\\n";
    cout << "1. Xoa dau\\n";
    cout << "2. Xoa duoi\\n";
    cout << "3. Xoa tai vi tri bat ky\\n";
    cout << "4. Xoa tai node ke ben node q\\n";
    do {
        int choose;
        cout << "Nhap lua chon: "; cin >> choose;
        switch (choose) {
            case 0:
                return 0;
                break;
            case 1:
                list->deleteFirst();
                cout << "Danh sach sau xoa dau: ";
                list->Travel();
                cout << "\\n=====\\n";
                break;
            case 2:
                list->deleteTail();
                cout << "Danh sach sau xoa duoi: ";
                list->Travel();
                cout << "\\n=====\\n";
                break;
            case 3:
                int vitri;
                cout << "Danh sach hien co " << list->countElement() << " phan tu.\\n";
                cout << "Nhap vi tri muon xoa: "; cin >> vitri;
                list->deletePositionK(vitri);
                cout << "Danh sach sau xoa tai vi tri " << vitri << ": ";
                list->Travel();
                cout << "\\n=====\\n";
                break;
            case 4:
                cout << "Nhap data cua phan tu truooc phan tu can xoa: ";
                int data; cin >> data;

```

```

        list->deleteBehindQ(data);
        cout << "Danh sach sau xoa :";
        list->Travel();
        cout << "\n=====\\n";
        break;
    default:
        break;
    }
} while (true);

delete e, list;
return 0;
}

```

Kết quả:

```

Danh sach hien co: 9      10      13      15      18
=====MENU XOA=====
0. Thoat
1. Xoa dau
2. Xoa duoi
3. Xoa tai vi tri bat ky
4. Xoa tai node ke ben node q
Nhap lua chon: 1
Danh sach sau xoa dau: 10      13      15      18
=====
Nhap lua chon: 2
Danh sach sau xoa duoi: 10      13      15
=====
Nhap lua chon: 3
Danh sach hien co 3 phan tu.
Nhap vi tri muon xoa: 2
Danh sach sau xoa tai vi tri 2: 10      15
=====
Nhap lua chon: 4
Nhap data cua phan tu truoc phan tu can xoa: 10
Danh sach sau xoa :10
=====
Nhap lua chon: 0
(Thoát chương trình)

```

2.5.3.7 Quản lý số lượng phần tử trong danh sách

Bổ sung hàm Travel ()

```

class linkedList {
...
public:
...
    int countElement() {
        element* p = head;
        int count = 0;
        while (p != nullptr) {
            count++;
            p = p->getPointer();
        }
        return count;
    }
};

```

2.5.3.8 Xóa danh sách liên kết đơn

➤ Bước 1: Trong khi việc duyệt danh sách chưa kết thúc, thực hiện:

▪ B1.1:

p = pHead;
pHead = pHead->pNext; // cập nhật pHead

▪ B1.2: Hủy p

➤ Bước 2: pTail = NULL; // bảo toàn tính nhất quán khi xâu rỗng

```
class linkedList {
...
public:
...
    void removeList() {
        element* p;
        while (head != nullptr) { //nếu danh sách không rỗng mới xóa
            p = head;
            setHead(p->getPointer()); //cho head trở đến node kế tiếp
            delete p; //hết vòng lặp head trở đến null
        }
        tail = nullptr;
        cout << "Danh sach da xoa thanh cong!\n";
    }
};
```

```
int main()
{
...
    do {
        int choose;
        cout << "Nhap lua chon: "; cin >> choose;
        switch (choose) {
            ...
            case 5:
                list->removeList();
                cout << "Danh sach sau xoa :";
                list->Travel();
                cout << "\n=====\\n";
                break;

            default:
                break;
        }
    } while (true);

    delete e, list;
    return 0;
}
```

Danh sach hien co: 9 10 13 15 18

=====MENU XOAA=====

0. Thoat

1. Xoa dau

2. Xoa duoi

3. Xoa tai vi tri bat ky

4. Xoa tai node ke ben node q

5. Xoa danh sach lien ket don.

Nhap lua chon: 5

Danh sach da xoa thanh cong!

Danh sach sau xoa :

=====

2.5.3.9 Sắp xếp danh sách liên kết đơn

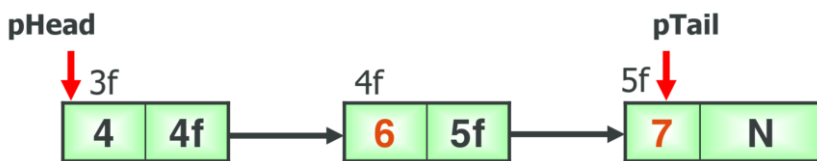
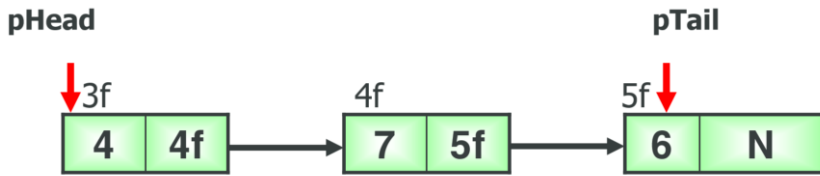
▪ Có 2 cách tiếp cận:

▪ Cách 1: Thay đổi thành phần dữ liệu của các nút.

▪ Ưu điểm: Cài đặt đơn giản, tương tự như sắp xếp mảng.

▪ Nhược điểm:

- Đòi hỏi thêm vùng nhớ khi hoán vị nội dung của 2 phần tử -> chỉ phù hợp với những DSLK có kích thước dữ liệu nhỏ.
- Khi kích thước dữ liệu lớn thì chi phí cho việc hoán vị thành phần dữ liệu cũng lớn.
- Làm cho thao tác sắp xếp chậm.



* **Ứng dụng sắp xếp chọn (selection sort)** (Có thể dùng giải thuật sắp xếp khác vì tương tự sắp xếp mảng)

- Chọn phần tử nhỏ nhất trong n phần tử ban đầu, đưa phần tử này về vị trí đúng là đầu tiên của dãy hiện hành. Sau đó không quan tâm đến nó nữa, xem dãy hiện hành chỉ còn n-1 phần tử của dãy ban đầu, bắt đầu từ vị trí thứ 2. Lặp lại quá trình trên cho dãy hiện hành đến khi dãy hiện hành chỉ còn một phần tử. Dãy ban đầu có n phần tử, vậy tóm tắt ý tưởng thuật toán là thực hiện n-1 lượt việc đưa phần tử nhỏ nhất trong dãy hiện hành về vị trí đúng ở đầu dãy.

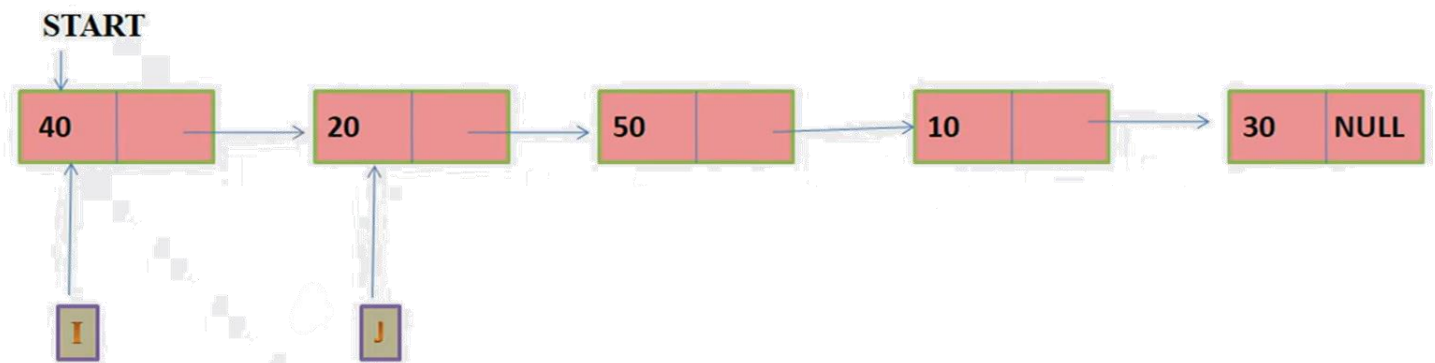
- Các bước thực hiện

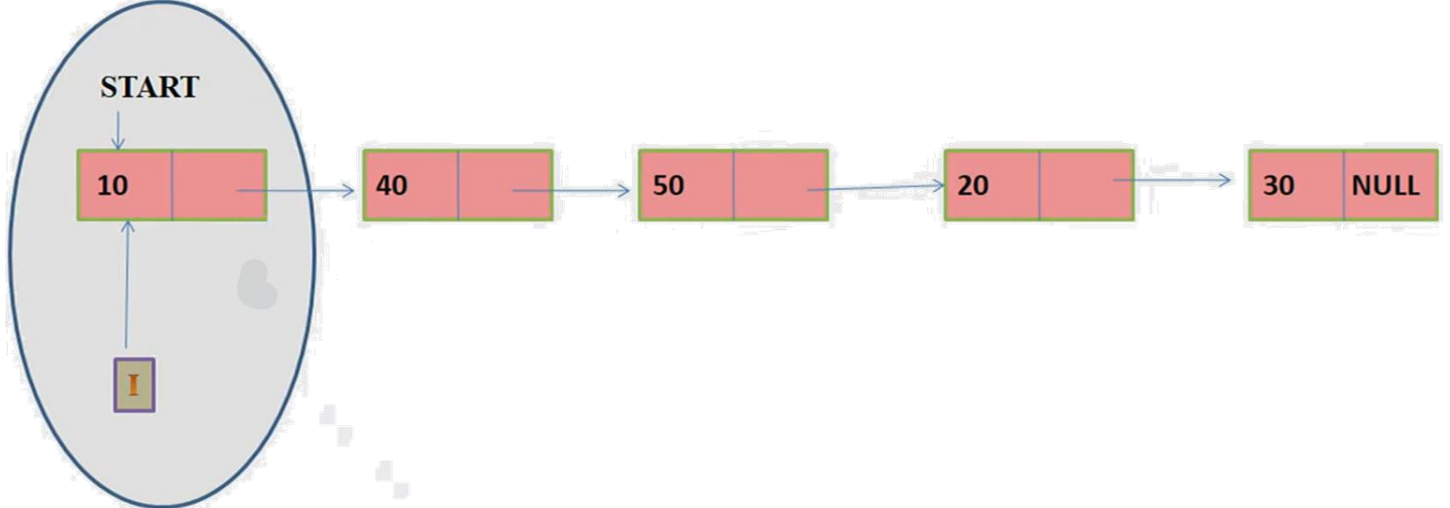
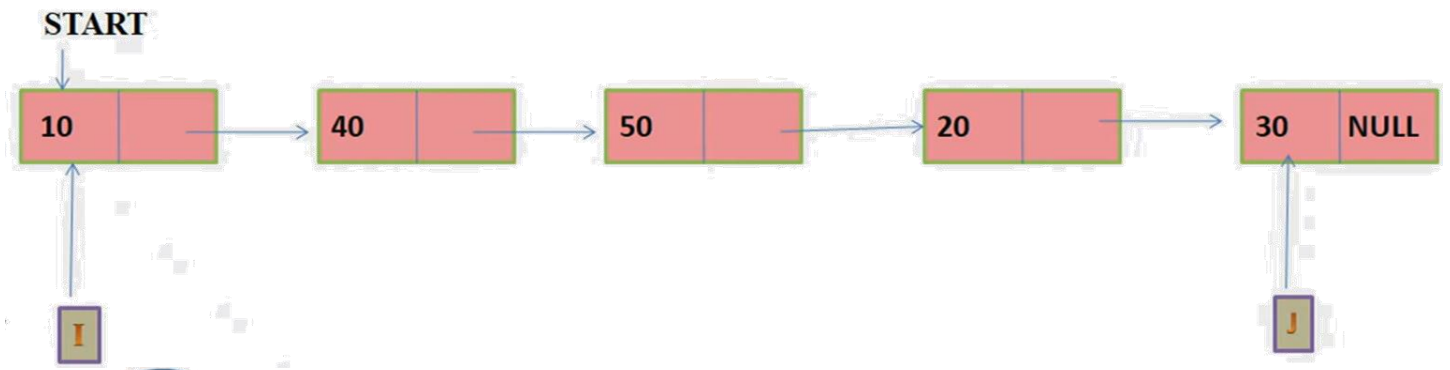
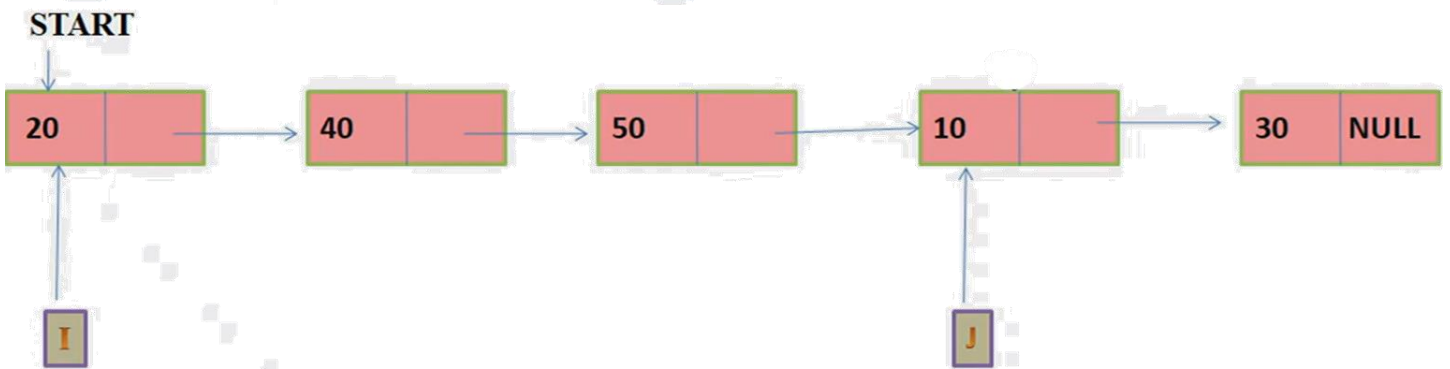
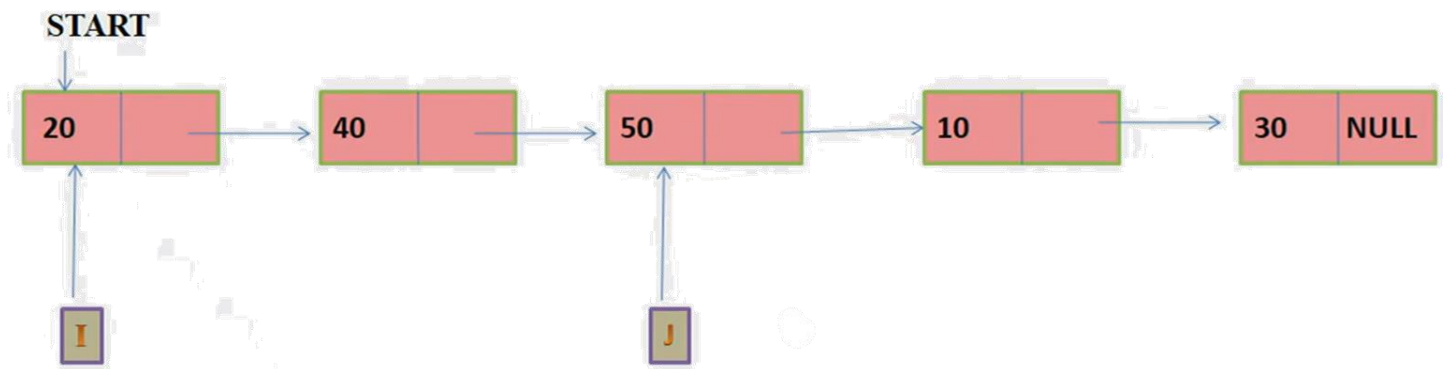
- Bước 1: $i=1$.
- Bước 2: Tìm phần tử $a_{[\min]}$ nhỏ nhất trong dãy hiện hành từ $a[i]$ đến $a[n]$.
- Bước 3: Hoán vị $a_{[\min]}$ và $a[i]$.
- Bước 4: Nếu $i \leq n-1$ thì $i=i+1$; Lặp lại bước 2.
- Ngược lại: Dừng. n-1 phần tử đã nằm đúng vị trí.

- Đánh giá

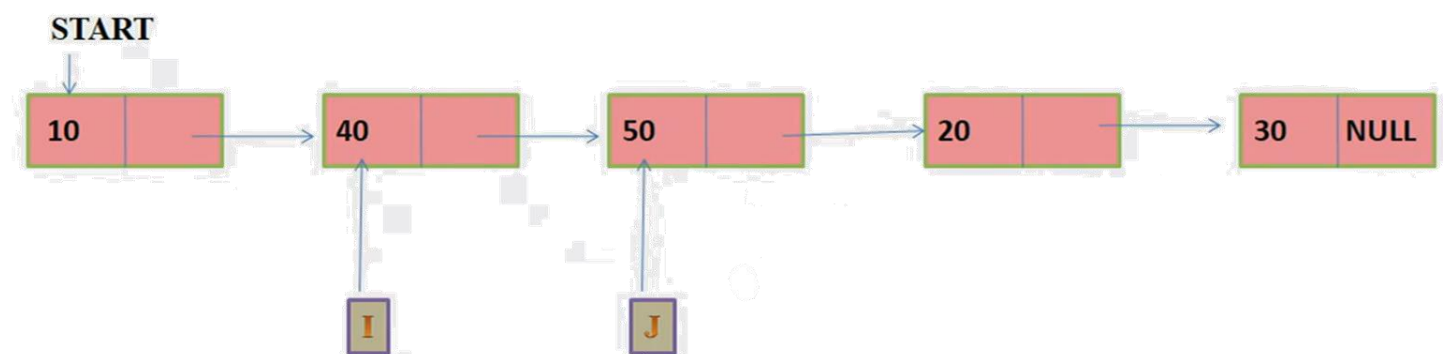
- Thuật toán ít phải đổi chỗ các phần tử nhất trong số các thuật toán sắp xếp (n lần hoán vị) nhưng có độ phức tạp so sánh là $O(n^2)$ ($n^2/2$ phép so sánh).
- Thuật toán tốn thời gian gần như bằng nhau đối với mảng đã được sắp xếp cũng như mảng chưa được sắp xếp.

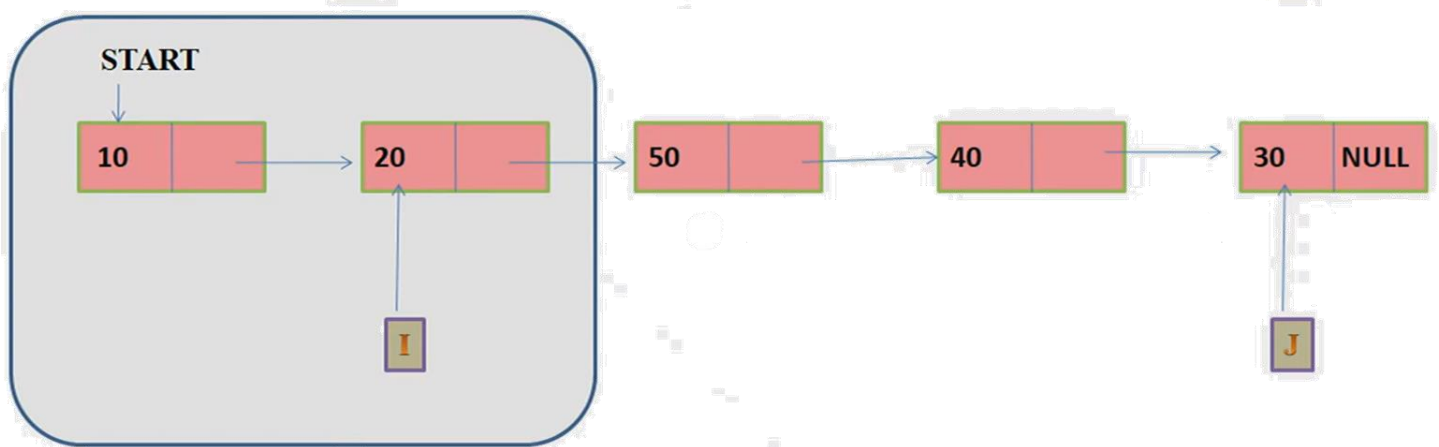
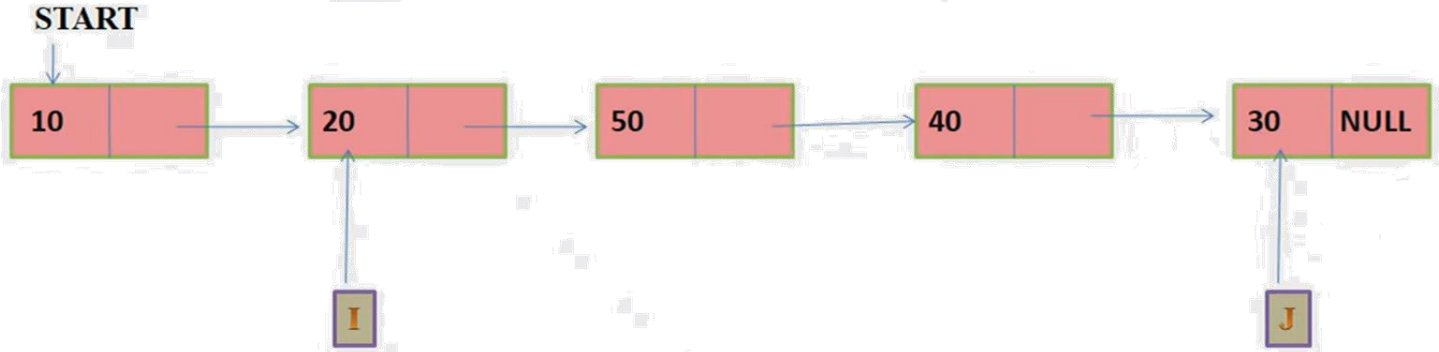
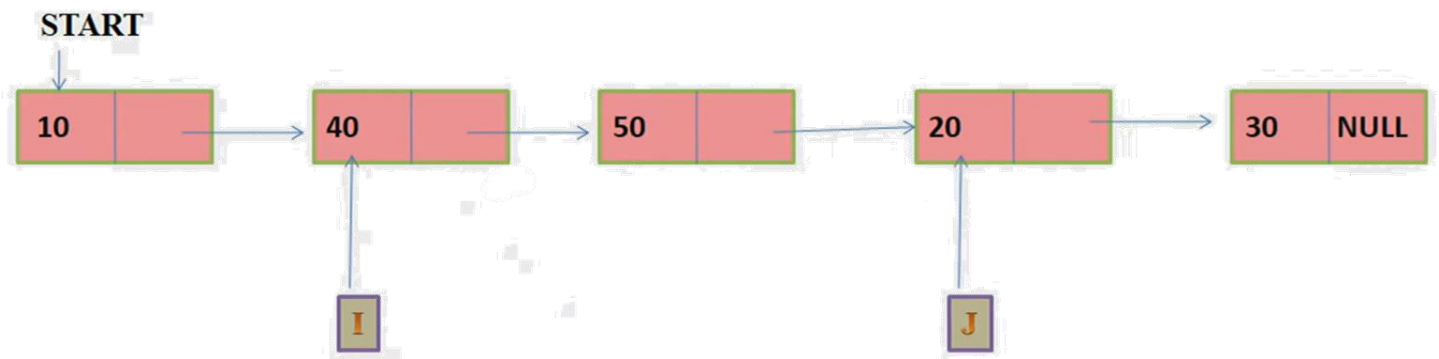
* Iteration 1



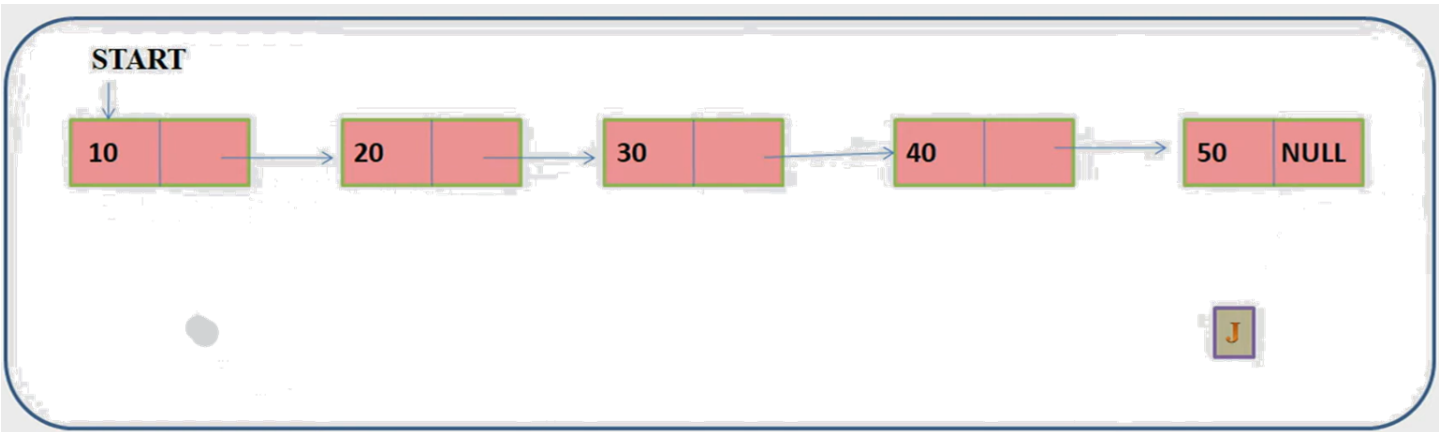


** Iteration 2*





... Iteration 5



```

class linkedList {
...
public:
...
    void sort() {
        for (element* i = head; i != nullptr; i = i->getPointer()) {
            element* min = i;
            for (element* j = i->getPointer(); j != nullptr; j = j->getPointer()) {
                if (min->getData() > j->getData()) {
                    min = j;
                }
            }
            //hoán vị
            int temp = min->getData();
            min->setData(i->getData());
            i->setData(temp);
        }
    }
};

```

```

int main()
{
    linkedList* list = new linkedList();

    element* e;
    e = new element(9); //e chứa địa chỉ của vùng nhớ được cấp phát bởi new
    list->insertTail(e);
    e = new element(7);
    list->insertTail(e);
    e = new element(13);
    list->insertTail(e);
    e = new element(5);
    list->insertTail(e);
    e = new element(8);
    list->insertTail(e);
    cout << "Danh sach hien co: ";
    list->Travel();
    cout << endl;

    cout << "Danh sach sau sap xep: ";
    list->sort();
    list->Travel();

    delete e, list;
    return 0;
}

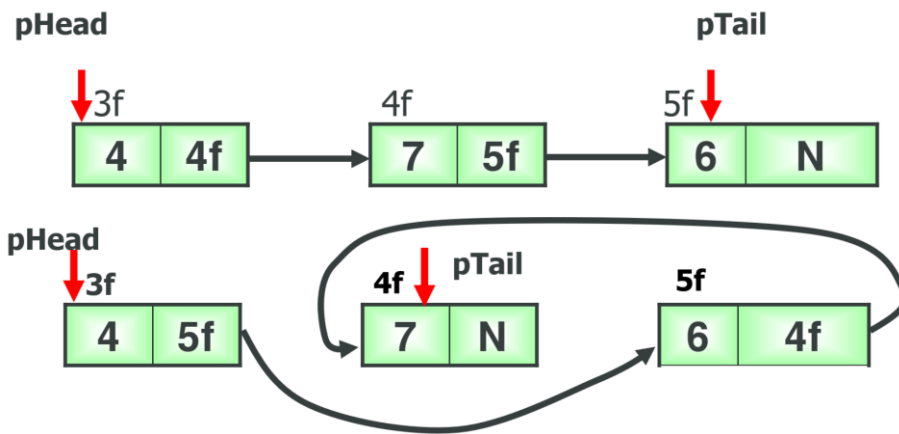
```

Kết quả:

Danh sach hien co: 9 7 13 5 8

Danh sach sau sap xep: 5 7 8 9 13

- **Cách 2:** Thay đổi thành phần con trỏ tiếp theo trong nút (thay đổi trình tự móc nối của các phần tử sao cho tạo lập nên được thứ tự mong muốn).
 - Ưu điểm:
 - *Kích thước của trường này không thay đổi, do đó không phụ thuộc vào kích thước bản chất dữ liệu lưu tại mỗi nút.*
 - *Thao tác sắp xếp nhanh.*
 - Nhược điểm: *Cài đặt phức tạp.*

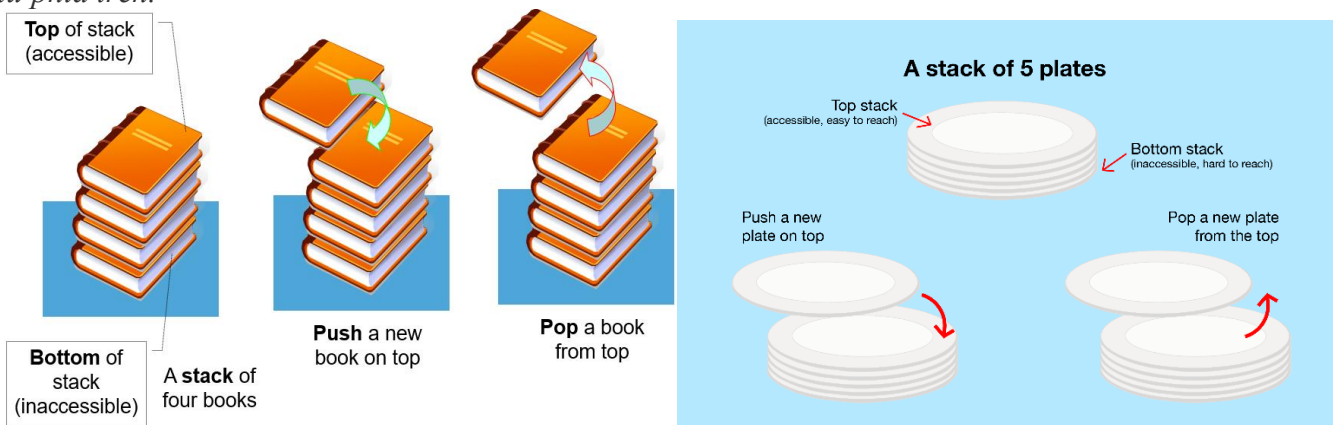


III. CÁC CẤU TRÚC ĐẶC BIỆT CỦA DANH SÁCH LIÊN KẾT ĐƠN

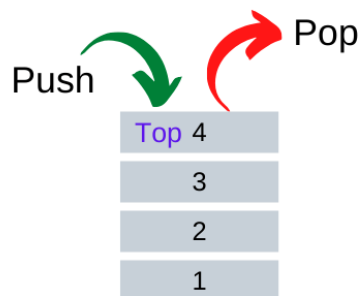
3.1 Stack (ngăn xếp)

Là một danh sách mà ta giới hạn việc thêm vào hoặc loại bỏ một phần tử chỉ thực hiện tại một đầu của danh sách, đầu này gọi là đỉnh (TOP) của ngăn xếp. Chính vì nguyên tắc này mà ngăn xếp còn được gọi là kiểu dữ liệu có nguyên tắc LIFO (Last In First Out – Vào sau ra trước)

Để dễ hình dung thì nó giống với hình ảnh 1 chồng sách, *tuy nhiên chồng sách này phải tuân theo một quy tắc đó là khi thêm một cuốn sách mới vào chồng thì phải thêm vào phía trên chồng sách và khi lấy sách ra cũng phải lấy từ phía trên.*



Stack



▪ Các thao tác trên ngăn xếp

- Thêm đối tượng vào stack. Push(o)
- Lấy đối tượng từ stack. Pop()
- Kiểm tra stack rỗng/ đầy hay không? isEmpty(); isFull()
- Lấy giá trị phần tử đỉnh của stack mà không hủy nó. Top()

3.1.1 Cài đặt bằng mảng 1 chiều

```
int n = 0, stack1[100];

void push(int x) {
    stack1[n] = x; //đưa phần tử vào đầu mảng
    ++n;
}
```

```

bool isEmpty() {
    if (n <= 0) return 1;
    return 0;
}
bool isFull() {
    if (n >= 100) return 1;
    return 0;
}

void pop() {
    if (n >= 1) {
        --n;
    }
}

int top() {
    if (n > 0) {
        return stack1[n - 1]; //lấy phần tử cuối mảng
    }
}

int size() {
    return n;
}

void TravelFromLast() {
    cout << "Duyet tu cuoi thi Stack hien chua: ";
    for (int i = 0; i < n; i++) {
        cout << stack1[i] << "\t";
    }
    cout << endl;
}

void TravelFromTop() {
    cout << "Duyet tu dau thi Stack hien chua: ";
    for (int i = n - 1; i >= 0; i--) {
        cout << stack1[i] << "\t";
    }
    cout << endl;
}

int main()
{
    //STACK AND QUEUE BANG MANG
    while (true) {
        cout << "=====\n";
        cout << "1. Push\n";
        cout << "2. Pop\n";
        cout << "3. Top\n";
        cout << "4. Size\n";
        cout << "5. IsFull\n";
        cout << "6. IsEmpty\n";
        cout << "7. Duyet tu day stack\n";
        cout << "8. Duyet tu dinh stack\n";
        cout << "0. Thoat\n";
        cout << "=====\n";

        int choice; cout << "Nhap lua chon: "; cin >> choice;
        switch (choice) {
            case 0:
                return 0;
                break;
            case 1:
                int x; cout << "Nhap x: "; cin >> x;
                push(x);
                break;
            case 2:
                pop();
                break;
            case 3:
                if (!isEmpty())

```

```

        cout << "Phan tu o dinh stack: " << top() << endl;
        cout << "Stack rong.\n";

        break;
    case 4:
        cout << "Stack co " << size() << " phan tu.\n";
        break;
    case 5:
        if (isFull()) cout << "Stack day. \n";
        else cout << "Stack chua day. \n";
        break;
    case 6:
        if (isEmpty()) cout << "Stack rong. \n";
        else cout << "Co phan tu trong stack.\n";
        break;
    case 7:
        TravelFromLast();
        break;
    case 8:
        TravelFromTop();
        break;
    default:
        break;
    }

}
return 0;
}

```

=====

1. Push
2. Pop
3. Top
4. Size
5. IsFull
6. IsEmpty
7. Duyệt tu day stack
8. Duyệt tu dinh stack
0. Thoat

=====

Nhap lua chon: 1

Nhap x: 1

=====

1. Push
2. Pop
3. Top
4. Size
5. IsFull
6. IsEmpty
7. Duyệt tu day stack
8. Duyệt tu dinh stack
0. Thoat

=====

Nhap lua chon: 1

Nhap x: 2

=====

1. Push
2. Pop
3. Top
4. Size
5. IsFull
6. IsEmpty

7. Duyệt tu day stack
8. Duyệt tu dinh stack
0. Thoat

=====

Nhap lua chon: 1

Nhap x: 3

=====

1. Push
2. Pop
3. Top
4. Size
5. IsFull
6. IsEmpty
7. Duyệt tu day stack
8. Duyệt tu dinh stack
0. Thoat

=====

Nhap lua chon: 1

Nhap x: 4

=====

1. Push
2. Pop
3. Top
4. Size
5. IsFull
6. IsEmpty
7. Duyệt tu day stack
8. Duyệt tu dinh stack
0. Thoat

=====

Nhap lua chon: 4

Stack co 4 phan tu.

=====

1. Push
2. Pop
3. Top
4. Size
5. IsFull
6. IsEmpty
7. Duyệt tu day stack
8. Duyệt tu dinh stack
0. Thoat

=====

Nhap lua chon: 7

Duyệt tu cuoi thi Stack hien chua: 1 2 3 4

=====

1. Push
2. Pop
3. Top
4. Size
5. IsFull
6. IsEmpty
7. Duyệt tu day stack
8. Duyệt tu dinh stack
0. Thoat

=====


```

Nhap lua chon: 8
Duyet tu dau thi Stack hien chua: 4      3      2      1
=====
1. Push
2. Pop
3. Top
4. Size
5. IsFull
6. IsEmpty
7. Duyet tu day stack
8. Duyet tu dinh stack
0. Thoat
=====
Nhap lua chon: 5
Stack chua day.
=====
1. Push
2. Pop
3. Top
4. Size
5. IsFull
6. IsEmpty
7. Duyet tu day stack
8. Duyet tu dinh stack
0. Thoat
=====
Nhap lua chon: 6
Co phan tu trong stack.
=====
1. Push
2. Pop
3. Top
4. Size
5. IsFull
6. IsEmpty
7. Duyet tu day stack
8. Duyet tu dinh stack
0. Thoat
=====
Nhap lua chon: 2
=====
1. Push
2. Pop
3. Top
4. Size
5. IsFull
6. IsEmpty
7. Duyet tu day stack
8. Duyet tu dinh stack
0. Thoat
=====
Nhap lua chon: 2
=====
1. Push
2. Pop
3. Top

```

4. Size
5. IsFull
6. IsEmpty
7. Duyệt tu day stack
8. Duyệt tu dinh stack
0. Thoat

=====

Nhap lua chon: 4

Stack co 2 phan tu.

=====

1. Push
2. Pop
3. Top
4. Size
5. IsFull
6. IsEmpty
7. Duyệt tu day stack
8. Duyệt tu dinh stack
0. Thoat

=====

Nhap lua chon: 2

=====

1. Push
2. Pop
3. Top
4. Size
5. IsFull
6. IsEmpty
7. Duyệt tu day stack
8. Duyệt tu dinh stack
0. Thoat

=====

Nhap lua chon: 2

=====

1. Push
2. Pop
3. Top
4. Size
5. IsFull
6. IsEmpty
7. Duyệt tu day stack
8. Duyệt tu dinh stack
0. Thoat

=====

Nhap lua chon: 4

Stack co 0 phan tu.

=====

1. Push
2. Pop
3. Top
4. Size
5. IsFull
6. IsEmpty
7. Duyệt tu day stack
8. Duyệt tu dinh stack
0. Thoat

=====

Nhap lua chon: 6

Stack rong.

=====

1. Push
2. Pop
3. Top
4. Size
5. IsFull
6. IsEmpty
7. Duyet tu day stack
8. Duyet tu dinh stack
0. Thoat

=====

Nhap lua chon: 0

3.1.2 Cài đặt bằng danh sách liên kết

```
class Stack1 {
private:
    linkedList *stack1;
    int soLuong; //quản lý số lượng phần tử trong stack
public:
    Stack1() { stack1 = new linkedList(); int soLuong = 0; }
    ~Stack1(){}
    void push(int x, int max){ //hàm chèn đầu
        if (!isFull(max)) {
            element* p = new element(x); //tạo một node có dữ liệu là x
            stack1->insertFirst(p);
            ++soLuong;
        }
        else
            cout << "Stack day. \n";
    }
    void pop() { //hàm xóa đầu
        if (!isEmpty()) {
            stack1->deleteFirst();
            --soLuong;
        }
        else
            cout << "Stack rong. \n";
    }
    void printStack() { //travel
        stack1->Travel();
    }
    int size() {
        return soLuong;
    }
    int Top() {
        return stack1->getHead()->getData();
    }
    bool isEmpty() {
        if (soLuong <= 0) return 1;
        return 0;
    }
    bool isFull(int maxSL) {
        if (soLuong >= maxSL) return 1;
        return 0;
    }
};

int main()
{
    //STACK AND QUEUE BANG DSLK DON
    Stack1* stack1 = new Stack1();
```

```

int max; cout << "Nhap so luong phan tu max trong stack: "; cin >> max;
while (true) {
    cout << "=====\n";
    cout << "1. Push\n";
    cout << "2. Pop\n";
    cout << "3. Top\n";
    cout << "4. Size\n";
    cout << "5. IsFull\n";
    cout << "6. IsEmpty\n";
    cout << "7. Duyet tu dinh stack\n";
    cout << "0. Thoat\n";
    cout << "=====\n";

    int choice; cout << "Nhap lua chon: "; cin >> choice;
    switch (choice) {
        case 0:
            return 0;
            break;
        case 1:
            int x; cout << "Nhap x: "; cin >> x;
            stack1->push(x, max);
            break;
        case 2:
            stack1->pop();
            break;
        case 3:
            if (!(stack1->isEmpty()))
                cout << "Phan tu o dinh stack: " << stack1->Top() << endl;
            else
                cout << "Stack rong.\n";
            break;
        case 4:
            cout << "Stack co " << stack1->size() << " phan tu.\n";
            break;
        case 5:
            if (stack1->isFull(max)) cout << "Stack day. \n";
            else cout << "Stack chua day. \n";
            break;
        case 6:
            if (stack1->isEmpty()) cout << "Stack rong. \n";
            else cout << "Co phan tu trong stack.\n";
            break;
        case 7:
            cout << "Stack hien co: ";
            stack1->printStack();
            cout << endl;
            break;
        default:
            break;
    }
}
delete stack1;
return 0;
}

```

Nhap so luong phan tu max trong stack: 100

=====

1. Push
2. Pop
3. Top
4. Size
5. IsFull
6. IsEmpty
7. Duyet tu dinh stack
0. Thoat

=====

Nhap lua chon: 1

Nhap x: 1

- 1. Push
- 2. Pop
- 3. Top
- 4. Size
- 5. IsFull
- 6. IsEmpty
- 7. Duyệt tu dinh stack
- 0. Thoat

Nhap lua chon: 1

Nhap x: 2

- 1. Push
- 2. Pop
- 3. Top
- 4. Size
- 5. IsFull
- 6. IsEmpty
- 7. Duyệt tu dinh stack
- 0. Thoat

Nhap lua chon: 1

Nhap x: 3

- 1. Push
- 2. Pop
- 3. Top
- 4. Size
- 5. IsFull
- 6. IsEmpty
- 7. Duyệt tu dinh stack
- 0. Thoat

Nhap lua chon: 1

Nhap x: 4

- 1. Push
- 2. Pop
- 3. Top
- 4. Size
- 5. IsFull
- 6. IsEmpty
- 7. Duyệt tu dinh stack
- 0. Thoat

Nhap lua chon: 3

Phan tu o dinh stack: 4

- 1. Push
- 2. Pop
- 3. Top
- 4. Size
- 5. IsFull
- 6. IsEmpty
- 7. Duyệt tu dinh stack

0. Thoat

Nhap lua chon: 7

Stack hien co: 4 3 2 1

1. Push

2. Pop

3. Top

4. Size

5. IsFull

6. IsEmpty

7. Duyet tu dinh stack

0. Thoat

Nhap lua chon: 4

Stack co 4 phan tu.

1. Push

2. Pop

3. Top

4. Size

5. IsFull

6. IsEmpty

7. Duyet tu dinh stack

0. Thoat

Nhap lua chon: 5

Stack chua day.

1. Push

2. Pop

3. Top

4. Size

5. IsFull

6. IsEmpty

7. Duyet tu dinh stack

0. Thoat

Nhap lua chon: 6

Co phan tu trong stack.

1. Push

2. Pop

3. Top

4. Size

5. IsFull

6. IsEmpty

7. Duyet tu dinh stack

0. Thoat

Nhap lua chon: 2

1. Push

2. Pop

3. Top

4. Size

5. IsFull

6. IsEmpty
7. Duyệt tu dinh stack
0. Thoat

=====

Nhap lua chon: 2

=====

1. Push
2. Pop
3. Top
4. Size
5. IsFull
6. IsEmpty
7. Duyệt tu dinh stack
0. Thoat

=====

Nhap lua chon: 2

=====

1. Push
2. Pop
3. Top
4. Size
5. IsFull
6. IsEmpty
7. Duyệt tu dinh stack
0. Thoat

=====

Nhap lua chon: 7
Stack hien co: 1

=====

1. Push
2. Pop
3. Top
4. Size
5. IsFull
6. IsEmpty
7. Duyệt tu dinh stack
0. Thoat

=====

Nhap lua chon: 2

=====

1. Push
2. Pop
3. Top
4. Size
5. IsFull
6. IsEmpty
7. Duyệt tu dinh stack
0. Thoat

=====

Nhap lua chon: 3
Stack rong.

=====

1. Push
2. Pop
3. Top
4. Size
5. IsFull

- 6. IsEmpty
- 7. Duyệt tu đỉnh stack
- 0. Thoat

Nhap lua chon: 6
Stack rong.

- 1. Push
- 2. Pop
- 3. Top
- 4. Size
- 5. IsFull
- 6. IsEmpty
- 7. Duyệt tu đỉnh stack
- 0. Thoat

Nhap lua chon: 4
Stack co 0 phan tu.

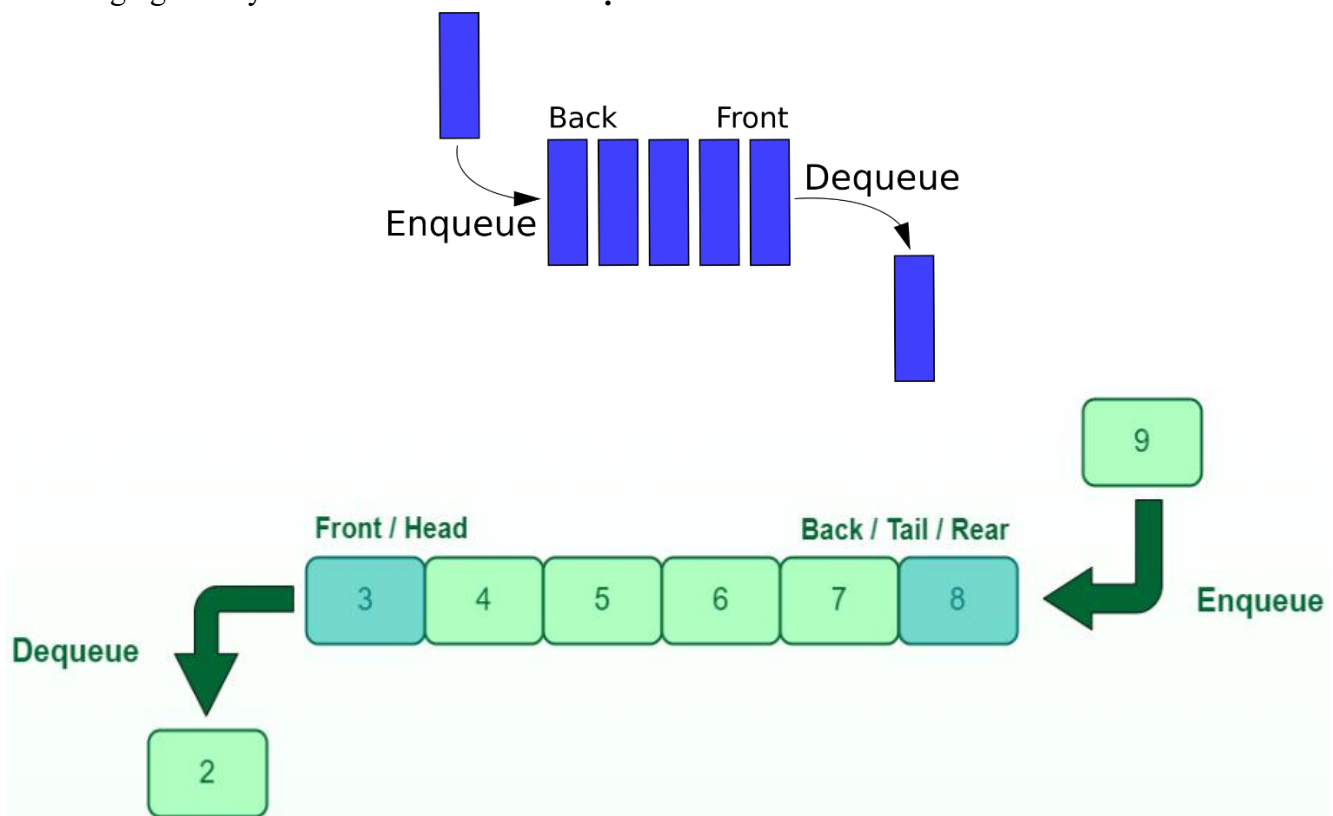
- 1. Push
- 2. Pop
- 3. Top
- 4. Size
- 5. IsFull
- 6. IsEmpty
- 7. Duyệt tu đỉnh stack
- 0. Thoat

Nhap lua chon: 0

3.2 Queue (hàng đợi)

Là danh sách làm việc theo cơ chế FIFO (First In First Out), tức việc thêm 1 đối tượng vào hàng đợi hay lấy 1 đối tượng ra khỏi hàng đợi thực hiện theo cơ chế “vào trước ra trước”.

Chúng ta có thể hình dung đó là hình ảnh một hàng người đang xếp hàng để mua pizza, và dĩ nhiên tính chất của hàng người này đó là **ai đến trước thì được mua trước**.



- **enqueue():** Thêm 1 phần tử vào Queue, tương tự với hàm thêm 1 phần tử vào cuối danh sách liên kết đơn.
- **dequeue():** Xóa 1 phần tử trong Queue, tương tự với hàm xóa 1 phần tử ở đầu danh sách liên kết đơn.
- **isEmpty()/ isFull():** Kiểm tra xem hàng đợi có rỗng hay không?
- **Front():** Trả về giá trị của phần tử nằm đầu hàng đợi mà không hủy nó.
- **Back():** Trả về giá trị của phần tử nằm cuối hàng đợi mà không hủy nó.

3.2.1 Cài đặt bằng mảng 1 chiều

```
int a[1000], maxN = 1000;
int n = 0;
int size() {
    return n;
}

bool isEmpty() {
    return n == 0;
}

bool isFull() {
    return n == maxN;
}

void enqueue(int x) {
    if (n == maxN) {
        cout << "Hang doi day.";
        return;
    }
    else
        a[n] = x; ++n;
}

void dequeue() {
    if (n == 0) {
        cout << "Hang doi rong.";
        return;
    }
    else {
        for (int i = 0; i < n - 1; i++) {
            a[i] = a[i + 1];
        }
        --n;
    }
}

int front() {
    return a[0];
}

int back() {
    return a[n - 1];
}

void travelQ() {
    cout << "Hang doi hien chua:\t";
    for (int j = 0; j < n; j++) {
        cout << a[j] << "\t";
    }
}

int main() {
    while (true) {
        cout << "\n===== \n";
        cout << "1. Enqueue\n";
        cout << "2. Dequeue\n";
        cout << "3. Front\n";
        cout << "4. Back\n";
        cout << "5. Duyet\n";
        cout << "6. KT day\n";
        cout << "7. KT rong\n";
    }
}
```

```

cout << "8. So luong phan tu\n";
cout << "0. Thoat\n";
cout << "=====\n";

int choice;
cout << "Nhap lua chon: "; cin >> choice;
switch (choice) {
case 0:
    return 0;
    break;
case 1:
    int dataQ;
    cout << "Nhap du lieu dua vao hang doi: "; cin >> dataQ;
    enqueue(dataQ);
    break;
case 2:
    dequeue();
    break;
case 3:
    cout << "Phan tu dau hang: " << front();
    break;
case 4:
    cout << "Phan tu cuoi hang: " << back();
    break;
case 5:
    travelQ();
    break;
case 6:
    if (isFull())
        cout << "Hang doi day.\n";
    else
        cout << "Hang doi chua day.\n";
    break;
case 7:
    if (isEmpty())
        cout << "Hang doi rong.\n";
    else
        cout << "Hang doi co phan tu.\n";
    break;
case 8:
    cout << "So luong phan tu trong hang doi: " << n;
    break;
default:
    break;
}
return 0;
}

```

- =====
1. Enqueue
 2. Dequeue
 3. Front
 4. Back
 5. Duyet
 6. KT day
 7. KT rong
 8. So luong phan tu
 0. Thoat
- =====

Nhap lua chon: 1
Nhap du lieu dua vao hang doi: 1

- =====
1. Enqueue
 2. Dequeue

3. Front
4. Back
5. Duyệt
6. KT day
7. KT rong
8. So luong phan tu
0. Thoat

=====

Nhap lua chon: 1

Nhap du lieu dua vao hang doi: 2

- =====
1. Enqueue
 2. Dequeue
 3. Front
 4. Back
 5. Duyệt
 6. KT day
 7. KT rong
 8. So luong phan tu
 0. Thoat

=====

Nhap lua chon: 1

Nhap du lieu dua vao hang doi: 3

- =====
1. Enqueue
 2. Dequeue
 3. Front
 4. Back
 5. Duyệt
 6. KT day
 7. KT rong
 8. So luong phan tu
 0. Thoat

=====

Nhap lua chon: 1

Nhap du lieu dua vao hang doi: 4

- =====
1. Enqueue
 2. Dequeue
 3. Front
 4. Back
 5. Duyệt
 6. KT day
 7. KT rong
 8. So luong phan tu
 0. Thoat

=====

Nhap lua chon: 8

So luong phan tu trong hang doi: 4

- =====
1. Enqueue
 2. Dequeue
 3. Front
 4. Back

- 5. Duyệt
- 6. KT day
- 7. KT rong
- 8. So luong phan tu
- 0. Thoat

=====

Nhap lua chon: 5

Hang doi hien chua: 1 2 3 4

=====

- 1. Enqueue
- 2. Dequeue
- 3. Front
- 4. Back
- 5. Duyệt
- 6. KT day
- 7. KT rong
- 8. So luong phan tu
- 0. Thoat

=====

Nhap lua chon: 3

Phan tu dau hang: 1

=====

- 1. Enqueue
- 2. Dequeue
- 3. Front
- 4. Back
- 5. Duyệt
- 6. KT day
- 7. KT rong
- 8. So luong phan tu
- 0. Thoat

=====

Nhap lua chon: 4

Phan tu cuoi hang: 4

=====

- 1. Enqueue
- 2. Dequeue
- 3. Front
- 4. Back
- 5. Duyệt
- 6. KT day
- 7. KT rong
- 8. So luong phan tu
- 0. Thoat

=====

Nhap lua chon: 6

Hang doi chua day.

=====

- 1. Enqueue
- 2. Dequeue
- 3. Front
- 4. Back
- 5. Duyệt
- 6. KT day
- 7. KT rong
- 8. So luong phan tu

0. Thoat

Nhap lua chon: 7
Hang doi co phan tu.

- 1. Enqueue
- 2. Dequeue
- 3. Front
- 4. Back
- 5. Duyet
- 6. KT day
- 7. KT rong
- 8. So luong phan tu
- 0. Thoat

Nhap lua chon: 2

- 1. Enqueue
- 2. Dequeue
- 3. Front
- 4. Back
- 5. Duyet
- 6. KT day
- 7. KT rong
- 8. So luong phan tu
- 0. Thoat

Nhap lua chon: 2

- 1. Enqueue
- 2. Dequeue
- 3. Front
- 4. Back
- 5. Duyet
- 6. KT day
- 7. KT rong
- 8. So luong phan tu
- 0. Thoat

Nhap lua chon: 2

- 1. Enqueue
- 2. Dequeue
- 3. Front
- 4. Back
- 5. Duyet
- 6. KT day
- 7. KT rong
- 8. So luong phan tu
- 0. Thoat

Nhap lua chon: 8
So luong phan tu trong hang doi: 1

- ```
=====
```
1. Enqueue
  2. Dequeue
  3. Front
  4. Back
  5. Duyệt
  6. KT day
  7. KT rong
  8. So luong phan tu
  0. Thoat
- ```
=====
```

Nhap lua chon: 5
Hang doi hien chua: 4

```
=====
```

1. Enqueue
 2. Dequeue
 3. Front
 4. Back
 5. Duyệt
 6. KT day
 7. KT rong
 8. So luong phan tu
 0. Thoat
- ```
=====
```

Nhap lua chon: 7  
Hang doi co phan tu.

```
=====
```

1. Enqueue
  2. Dequeue
  3. Front
  4. Back
  5. Duyệt
  6. KT day
  7. KT rong
  8. So luong phan tu
  0. Thoat
- ```
=====
```

Nhap lua chon: 0

3.2.2 Cài đặt bằng danh sách liên kết đơn

```
class Queue2 {  
private:  
    LinkedList *queue2;  
    int soLuong2;  
public:  
    Queue2() {  
        queue2 = new LinkedList();  
        soLuong2 = 0;  
    }  
    ~Queue2() {};  
  
    bool isEmpty() {  
        if (soLuong2 <= 0) return 1;  
        return 0;  
    }  
  
    bool isFull() {  
        if (soLuong2 >= 100) return 1;  
        return 0;  
    }  
}
```

```

void enqueue(int x) {
    if (!isFull()) {
        element* p = new element(x);
        queue2->insertTail(p);
        ++soLuong2;
    }
    else
        cout << "Hang doi da day.\n";
}

void dequeue() {
    if (!isEmpty()) {
        queue2->deleteFirst();
        --soLuong2;
    }
    else
        cout << "Hang doi rong. Khong co gi de xoa.\n";
}

void front() {
    if (!isEmpty()) {
        cout << queue2->getHead()->getData();
    }
    else
        cout << "Hang doi rong.\n";
}

void back(){
    if (!isEmpty()) {
        cout << queue2->getTail()->getData();
    }
    else
        cout << "Hang doi rong.\n";
}

int GetsoLuong2() {
    return soLuong2;
}

void DuyetQ() {
    queue2->Travel();
}
};

int main() {
    Queue2 *queue2 = new Queue2();
    while (true) {
        cout << "\n===== \n";
        cout << "1. Enqueue\n";
        cout << "2. Dequeue\n";
        cout << "3. Front\n";
        cout << "4. Back\n";
        cout << "5. Duyet\n";
        cout << "6. KT day\n";
        cout << "7. KT rong\n";
        cout << "8. So luong phan tu\n";
        cout << "0. Thoat\n";
        cout << "===== \n";

        int choice;
        cout << "Nhap lua chon: "; cin >> choice;
        switch (choice) {
            case 0:
                return 0;
                break;
            case 1:
                int dataQ;
                cout << "Nhap du lieu dua vao hang doi: "; cin >> dataQ;
                queue2->enqueue(dataQ);
                break;
            case 2:

```

```

        queue2->deQueue();
        break;
    case 3:
        cout << "Phan tu dau hang: "; queue2->front();
        break;
    case 4:
        cout << "Phan tu cuoi hang: "; queue2->back();
        break;
    case 5:
        queue2->DuyetQ();
        break;
    case 6:
        if (queue2->isFull())
            cout << "Hang doi day.\n";
        else
            cout << "Hang doi chua day.\n";
        break;
    case 7:
        if (queue2->isEmpty())
            cout << "Hang doi rong.\n";
        else
            cout << "Hang doi co phan tu.\n";
        break;
    case 8:
        cout << "So luong phan tu trong hang doi: " << queue2->GetsoLuong2();
        break;
    default:
        cout << "Lua chon khong dung. Moi chon lai.";
        break;
    }
}
return 0;
}

```

- =====
1. Enqueue
 2. Dequeue
 3. Front
 4. Back
 5. Duyet
 6. KT day
 7. KT rong
 8. So luong phan tu
 0. Thoat
- =====

Nhap lua chon: 1

Nhap du lieu dua vao hang doi: 1

- =====
1. Enqueue
 2. Dequeue
 3. Front
 4. Back
 5. Duyet
 6. KT day
 7. KT rong
 8. So luong phan tu
 0. Thoat
- =====

Nhap lua chon: 1

Nhap du lieu dua vao hang doi: 2

=====

1. Enqueue
2. Dequeue
3. Front
4. Back
5. Duyệt
6. KT day
7. KT rong
8. So luong phan tu
0. Thoat

Nhap lua chon: 1

Nhap du lieu dua vao hang doi: 3

1. Enqueue
2. Dequeue
3. Front
4. Back
5. Duyệt
6. KT day
7. KT rong
8. So luong phan tu
0. Thoat

Nhap lua chon: 1

Nhap du lieu dua vao hang doi: 4

1. Enqueue
2. Dequeue
3. Front
4. Back
5. Duyệt
6. KT day
7. KT rong
8. So luong phan tu
0. Thoat

Nhap lua chon: 5

1 2 3 4

1. Enqueue
2. Dequeue
3. Front
4. Back
5. Duyệt
6. KT day
7. KT rong
8. So luong phan tu
0. Thoat

Nhap lua chon: 4

Phan tu cuoi hang: 4

1. Enqueue
2. Dequeue
3. Front

- 4. Back
- 5. Duyệt
- 6. KT day
- 7. KT rong
- 8. So luong phan tu
- 0. Thoat

=====

Nhap lua chon: 3
Phan tu dau hang: 1

=====

- 1. Enqueue
- 2. Dequeue
- 3. Front
- 4. Back
- 5. Duyệt
- 6. KT day
- 7. KT rong
- 8. So luong phan tu
- 0. Thoat

=====

Nhap lua chon: 8
So luong phan tu trong hang doi: 4

=====

- 1. Enqueue
- 2. Dequeue
- 3. Front
- 4. Back
- 5. Duyệt
- 6. KT day
- 7. KT rong
- 8. So luong phan tu
- 0. Thoat

=====

Nhap lua chon: 2

=====

- 1. Enqueue
- 2. Dequeue
- 3. Front
- 4. Back
- 5. Duyệt
- 6. KT day
- 7. KT rong
- 8. So luong phan tu
- 0. Thoat

=====

Nhap lua chon: 5
2 3 4

=====

- 1. Enqueue
- 2. Dequeue
- 3. Front
- 4. Back
- 5. Duyệt
- 6. KT day
- 7. KT rong
- 8. So luong phan tu

0. Thoat

Nhap lua chon: 7

Hang doi co phan tu.

1. Enqueue

2. Dequeue

3. Front

4. Back

5. Duyet

6. KT day

7. KT rong

8. So luong phan tu

0. Thoat

Nhap lua chon: 8

So luong phan tu trong hang doi: 3

1. Enqueue

2. Dequeue

3. Front

4. Back

5. Duyet

6. KT day

7. KT rong

8. So luong phan tu

0. Thoat

Nhap lua chon: 0